

PORTFOLIO

신입 게임 서버 프로그래머 윤성열

윤성열

syyundev@gmail.com

010-9248-6948



윤성열

신입 게임 서버 프로그래머

“안정적이고 확장 가능한
게임 서버를 설계하고 구현하며, 성능 개선과
문제 해결에 집중합니다.”

프로필



2002.05.12



서울특별시 양천구



syyundev@gmail.com



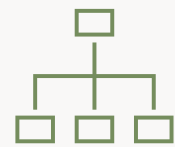
010-9248-6948



<https://github.com/syyundev>



보유 역량



서버 프레임워크 설계

확장성과 안정성을 고려한 게임
서버 구조 설계 및 구현



네트워크 프로그래밍

TCP/IP 기반 기본적인 네트워크
프로그래밍 및 IOCP, RIO 기반
고성능 비동기 네트워크 서버 개발



DB 연동 및 관리

MSSQL를 이용한 데이터 설계
및 ODBC 연동



문제해결 및 최적화

병목 분석을 통한 성능 개선 및
안정성 향상 경험



대표 프로젝트

01

EisenValor

졸업작품(팀 프로젝트)



2025.06.30 ~ 2026.06.22

- IOCP 로비서버와 RIO 게임서버 구현
- PVP/PVE 전투, BT/FSM을 이용한 AI
- FlatBuffers 라이브러리를 이용한
프로토콜 설계 및 구현
- DB를 통한 유저 정보 관리

C++20

IOCP/RIO

MSSQL

Git

02

Snake Game

팀 텀프로젝트



2025.10.31 ~ 2025.12.02

- TCP/IP 기반 논블로킹 멀티플레이 서버 구현
- 게임 도중 입/퇴장 및 이동 동기화
- 핵심적인 게임 콘텐츠 구현
- 프로토콜 설계 및 구현

C++20

TCP/IP

OOP

Git

03

SIMPLEST MMORPG

개인 텀프로젝트



2025.05.21 ~ 2025.06.15

- IOCP를 사용한 멀티 쓰레드 서버 구현
- 시야처리 및 공간분할 구현
- A* 알고리즘을 이용한 몬스터 길 찾기 구현
- 더미 클라이언트를 통한 동접 테스트

C++20

IOCP

MSSQL

Git



보유 기술

언어

C, C++20

네트워크

Winsock, TCP/IP, IOCP, RIO

데이터베이스

MSSQL

도구

Visual Studio, SSMS, Git/Github



학력 및 활동

학력사항

- 2021.02.04 서울 백암고등학교 졸업
- 2021.03.02 한국공학대학교(구 한국산업기술대학교) 게임공학과 입학
- 2027.02.19 한국공학대학교 게임공학과 졸업 예정

활동사항

- 2022.03.02 한국공학대학교 게임공학과 학술 소모임 WARP 운영진 및 C/C++ 멘토
- 2023.03.01

Project 01. EisenValor

2026학년도 1학기 종합설계1



IOCP 기반 로비서버와 RIO 기반 MO 게임서버를 제작하였습니다.

이동, 전투, AI등 모든 게임 콘텐츠 로직을 구현하고 DB를 연동하였습니다.

또한, 게임에 필요한 모든 프로토콜을 설계하고 구현하였습니다.

게임 장르	3D MO 전투 액션
개발 기간	2025.06.30 ~ 2026.06.22
개발 인원	클라이언트 프로그래머 2인, 서버 프로그래머 1인
역할 및 구현 내용	서버 프로그래머

- OOP기반의 서버 프레임워크 제작
 - IOCP 기반 로비서버 구현
 - 로비에서 방 생성/참여 및 Bot 추가 시스템 구현
 - DB를 통해 유저 정보 관리
 - RIO 기반 MO 게임서버 구현
 - PVP/PVE 전투, 점령, 회복 등 모든 게임 콘텐츠 구현
 - RecastDetour 라이브러리를 사용한 Navigation Mesh 기반 길 찾기
 - Hybrid FSM-BT 기반의 장수 AI, FSM 기반의 병사 AI 구현
- FlatBuffers 라이브러리를 사용한 프로토콜 제작, 패킷 핸들러 제작
- 클라이언트 사이드 네트워크 로직 구현

개발 언어

C++20

개발 도구 & 라이브러리

Visual Studio 2022, SSMS 21, Git/Github, Intel TBB, RecastDetour, Flatbuffers, Rapidjson, Claude Code CLI(Opus 4.8), Codex(GPT-5.5 High)

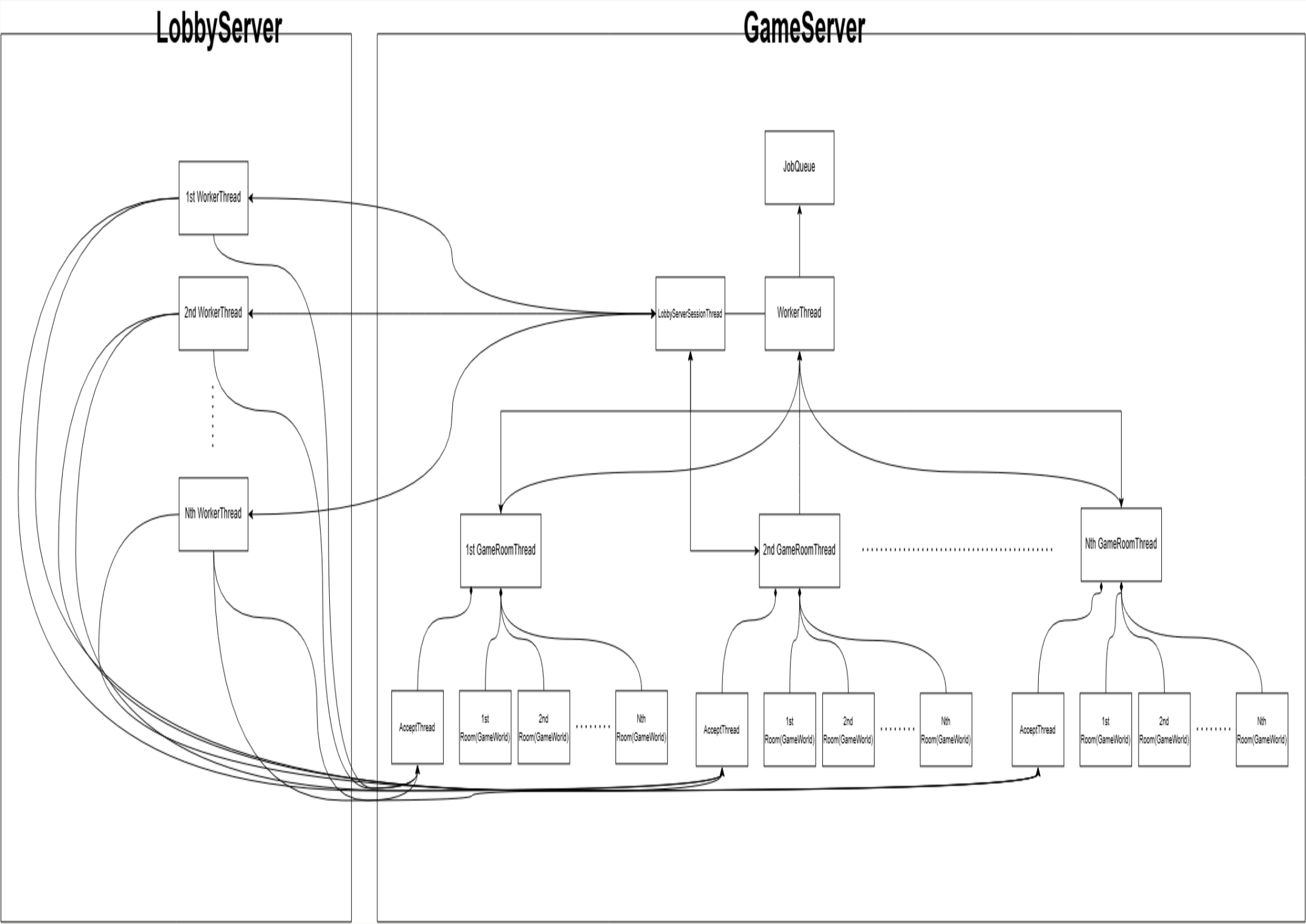
Github 주소

<https://github.com/yunsang1ee/EisenValor>



EisenValor– Server Architecture

서버 구조 설계



```
void GameServerEngine::GameWorldThread::Run(const std::stop_token st)
{
    MANAGER(ThreadManager)->EnqueueTask([this](const std::stop_token st)
    {
        m_acceptThread->Run(st);
    });

    auto last{ std::chrono::high_resolution_clock::now() };
    while(false == st.stop_requested()) {
        const auto now{ std::chrono::high_resolution_clock::now() };

        const std::chrono::duration<float> elapsed{ now - last };
        const float dt{ elapsed.count() };
        m_dt += dt;
        last = now;

        FlushJobQueue();

        m_ioCore->ProcessIO(); // 내가 관리하는 게임 월드 안에 있는 세션들의 Send/Recv (패킷 받고 보내기)

        // 월드에서 무한루프돌면 I/O 안됨 조심
        for(const auto& [id, world] : m_worlds) // 내가 관리하는 게임 월드
            world->Update(dt);

        ProcessPendingDestroyWorlds();
    }
}
```

로그인, 채팅, 게임 방 생성 및 참여 같은 기능은 비교적 짧고 불규칙적으로 발생합니다. 반면 이동, 공격, 충돌 검사, 상태 갱신, 게임 이벤트 처리와 같은 게임 로직들은 일정한 주기에 따라 지속적으로 수행되며 하나의 요청을 처리하는데 필요한 연산량도 상대적으로 큼니다.

이처럼 처리 방식이 다른 기능을 하나의 프로세스에서 함께 수행할 경우, 특정 기능의 부하가 다른 기능의 직접적인 영향을 줄 수 있다고 생각했습니다. 예를 들어, 게임 월드에서 전투나 충돌 검사와 같은 연산이 급격하게 증가하면 로그인이나 로비 요청에 대한 응답이 지연될 수 있습니다. 반대로 로비에 순간적으로 많은 사용자가 접속하거나 다수의 게임 방 생성 요청이 발생하면, 게임 월드의 업데이트 주기가 밀릴 수 있습니다.

이러한 상호 간섭을 최소화하고 각 서비스의 안정성을 확보하기 위해서 기능의 성격에 따라 로비서버와 게임 서버를 독립적인 프로세스로 분리하였습니다.

게임 서버는 여러 개의 GameWorldThread로 구성되며, 각 쓰레드는 자신에게 할당된 여러 GameRoom의 게임 로직과 Room안에 있는 세션들의 네트워크 I/O를 담당합니다. 각 Room은 GameWorldThread에 고정 배정되어 네트워크 I/O, 이동, 공격, 충돌 검사, 상태 갱신, 등의 작업을 순차적으로 처리합니다. 게임룸을 하나의 쓰레드에서 실행하고, 하나의 세션은 한 쓰레드가 계속 처리하도록 하여 RIO 최적화에 유리하게 구현했습니다. 이를 통해 게임 상태의 일관성을 유지하고 불필요한 락과 쓰레드 동기화 비용을 줄였으며, 여러 쓰레드가 서로 다른 게임 방을 병렬로 처리하므로 CPU자원을 효율적으로 활용하고 특정 방의 부하가 다른 방에 미치는 영향을 최소화했습니다.

EisenValor – Implementation details

주요 구현 내용

```
class RIORingBuffer {
protected:
    RIO_BUFFERID    m_bufferId;
    char*           m_buffer;
    uint32          m_capacity;
    uint32          m_usedSize;
    uint32          m_readOffset;
    uint32          m_writeOffset;

public:
    explicit RIORingBuffer(const uint32 capacity = 655360);
    virtual ~RIORingBuffer();

public:
    bool RegisterBuffer(const RIO_EXTENSION_FUNCTION_TABLE& rioFuncTable);
    void DeregisterBuffer(const RIO_EXTENSION_FUNCTION_TABLE& rioFuncTable);
    uint32 GetFreeSize() const { return m_capacity - m_usedSize; }
    uint32 GetUsedSize() const { return m_usedSize; }

    uint32 GetWriteOffset() const { return m_writeOffset; }

    RIO_BUF GetWriteRioBuf() const { return { m_bufferId, (ULONG)m_writeOffset, (ULONG)GetContiguousFreeSize() }; }

    bool OnWrite(const uint32 len);

    bool OnRead(const uint32 len);

    void AdjustPos();

    char* GetReadPos() { return m_buffer + m_readOffset; }

    uint32 GetContiguousReadSize() const;
    uint32 GetContiguousFreeSize() const;

    char* GetReadCursor() { return GetReadPos(); }

    void CleanBuffer();

    RIO_BUFFERID GetID() const { return m_bufferId; }
    uint32 GetCapacity() const { return m_capacity; }

private:
    void Alloc();
    void Free();
};
```

1) RIORingBuffer.h

```
void GameServerEngine::RIORingBuffer::Alloc()
{
    m_buffer = reinterpret_cast<char*>(VirtualAllocEx(GetCurrentProcess(), 0, m_capacity, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE));

    if(nullptr == m_buffer)
        LOG_ERROR("RIORingBuffer Alloc Failed!");
}
```

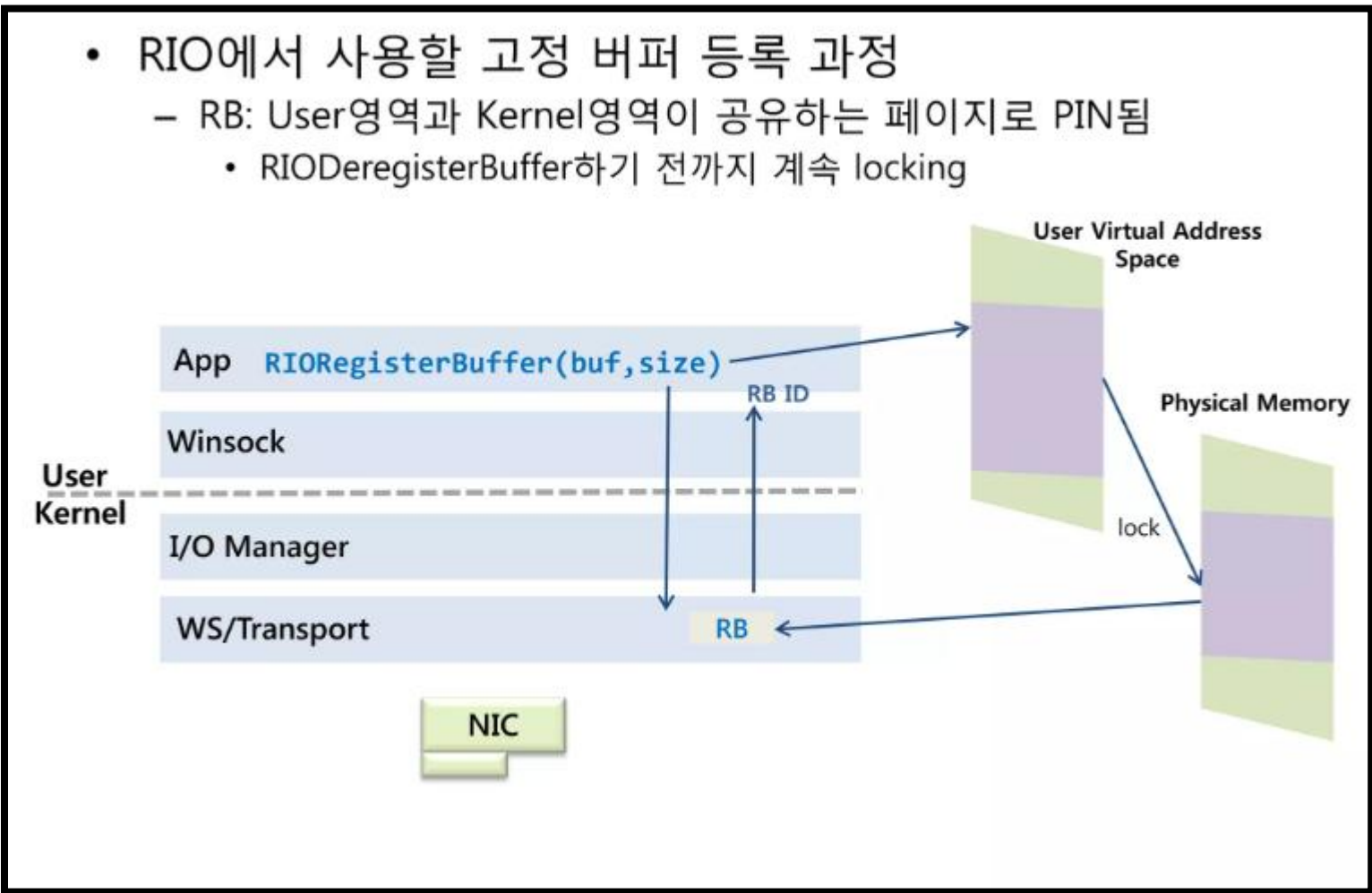


사진 출처: <https://www.slideshare.net/slideshow/windows-registered-io-rio/24250865>

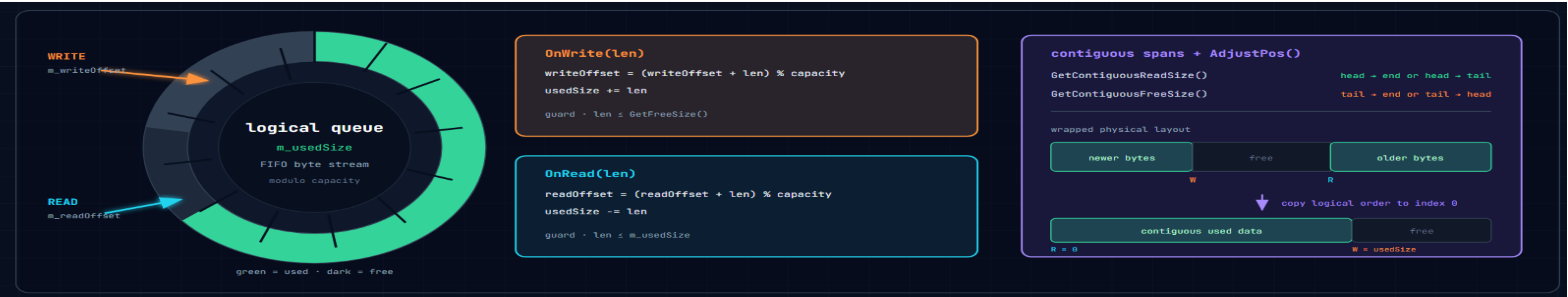
1) RIO 기반 Page-Locked Ring Buffer 설계

일반적인 RingBuffer의 순환형 메모리 관리 구조를 RIO에 맞게 확장한 RIORingBuffer를 구현했습니다.

VirtualAllocEx로 확보한 사용자 영역 버퍼를

RIORegisterBuffer에 등록하면 해당 버퍼를 구성하는 가상 메모리 페이지가 물리 메모리에 고정(PIN/Lock)됩니다. 이후 매 I/O 요청마다 사용자 버퍼의 유효성을 다시 검사하고 페이지를 잠그는 대신, RIO_BUF 구조체의 RIO_BUFFERID와 Offset, Length를 통해 버퍼 구간을 참조합니다.

이렇게 함으로써 매 번의 I/O마다 Page-lock/unlock 하는 오버헤드를 줄였습니다.



EisenValor – Implementation details

주요 구현 내용

```
enum class BEHAVIOR_NODE_STATUS {
    SUCCESS,
    FAIL,
    RUNNING,
};

class BehaviorNode {
public:
    virtual ~BehaviorNode() = default;

public:
    virtual void SetOwner(std::shared_ptr<General> const owner) { m_owner = owner; }
    virtual BEHAVIOR_NODE_STATUS Execute(const float dt) abstract;
    virtual void Reset() {}

public:
    std::shared_ptr<General> GetOwner() const { return m_owner.lock(); }

protected:
    std::weak_ptr<General> m_owner;
};

// 여러 자식을 가지고 있는 노드
class CompositeNode : public BehaviorNode {
public:
    virtual void Reset() override;
    virtual void SetOwner(std::shared_ptr<General> const owner) override;

public:
    void AddChild(std::unique_ptr<BehaviorNode> child);

protected:
    std::vector<std::unique_ptr<BehaviorNode>> m_children;
};

// 모든 자식이 순서대로 성공해야 전체가 성공
class SequenceNode : public CompositeNode {
public:
    virtual BEHAVIOR_NODE_STATUS Execute(const float dt) override;
    virtual void Reset() override { m_currentIndex = 0; CompositeNode::Reset(); }

private:
    size_t m_currentIndex = 0;
};

// 자식 중 하나라도 성공하면 전체가 성공
class SelectorNode : public CompositeNode {
public:
    virtual BEHAVIOR_NODE_STATUS Execute(const float dt) override;
    virtual void Reset() override { m_currentIndex = 0; CompositeNode::Reset(); }

private:
    size_t m_currentIndex = 0;
};
```

2) BehaviorNode.h

```
// =====
//                               IDLE
// =====
GameServer::Contents::GeneralIdleState::GeneralIdleState(const std::shared_ptr<General>& owner)
: GeneralState{ FB_ENUMS::GENERAL_STATE_TYPE_IDLE }

{
    auto rootSelector = std::make_unique<GameServer::Contents::SelectorNode>();

    // 0) 최초 생성 직후 2초 대기 (1회성, 이후 IDLE 재진입에는 영향 없음)
    rootSelector->AddChild(std::make_unique<GameServer::Contents::WaitOnce>(2.f));

    // 1) 적 감지 → 전투 진입 (Soldier 타겟이면 NEUTRAL, 그 외엔 COMBAT)
    {
        auto seq = std::make_unique<GameServer::Contents::SequenceNode>();
        seq->AddChild(std::make_unique<GameServer::Contents::FindEnemy>());
        seq->AddChild(std::make_unique<GameServer::Contents::SetStanceByTarget>());
        seq->AddChild(std::make_unique<GameServer::Contents::ChangeState>(FB_ENUMS::GENERAL_STATE_TYPE_WALK));
        rootSelector->AddChild(std::move(seq));
    }

    // 2) 아직 점령되지 않은 점령지 아니면 그대로 대기 (점령 진행)
    rootSelector->AddChild(std::make_unique<GameServer::Contents::IsInUnoccupiedZone>());

    // 3) 모든 점령지가 점령된 상태에서 이미 어떤 점령지 안에 있으면 굳이 다른 점령지로
    // 옮길 필요가 없으므로 그대로 대기 (IDLE ← RUN flap 방지).
    {
        auto seq = std::make_unique<GameServer::Contents::SequenceNode>();
        seq->AddChild(std::make_unique<GameServer::Contents::AreAllZonesOccupied>());
        seq->AddChild(std::make_unique<GameServer::Contents::IsInOccupationZone>());
        rootSelector->AddChild(std::move(seq));
    }

    // 4) 점령되지 않은 다른 점령지로 이동 (RUN 진입 전 기본 속도 복귀)
    // - 점령지 바뀌거나, 이미 점령된 점령지에 머무는 경우 모두 여기로 흐른다.
    // - MoveToOZ가 점령되지 않은 점령지가 없으면 전체 점령지 중 랜덤을 선택한다.
    {
        auto seq = std::make_unique<GameServer::Contents::SequenceNode>();
        seq->AddChild(std::make_unique<GameServer::Contents::SetMaxSpeed>(3.f));
        seq->AddChild(std::make_unique<GameServer::Contents::MoveToOZ>());
        seq->AddChild(std::make_unique<GameServer::Contents::ChangeState>(FB_ENUMS::GENERAL_STATE_TYPE_RUN));
        rootSelector->AddChild(std::move(seq));
    }

    rootSelector->SetOwner(owner);
    m_root = std::move(rootSelector);
}
```

2) GeneralNodes.cpp

```
class PacketHandler {
    using PacketHandlerFunc = bool (*)(const std::shared_ptr<GameServerEngine::PacketSession>&, std::span<const char>);
public:
    PacketHandler();
    virtual ~PacketHandler();

public:
    virtual void Init() abstract;

public:
    bool HandlePacket(const std::shared_ptr<GameServerEngine::PacketSession>& session, std::span<const char> buffer);

public:
    static bool Handle_INVALID(const std::shared_ptr<GameServerEngine::PacketSession>& session, std::span<const char> buffer) { return false; }

    template<typename PacketType, typename HandleFunc>
    static bool HandlePacket(const HandleFunc handleFunc, const std::shared_ptr<GameServerEngine::PacketSession>& session, std::span<const char> buffer)
    {
        flatbuffers::Verifier verifier{
            reinterpret_cast<const uint8_t*>(buffer.data()),
            buffer.size()
        };

        if(false == verifier.VerifyBuffer<PacketType>())
            return false;

        const PacketType* const packet = flatbuffers::GetRoot<PacketType>(buffer.data());
        if(nullptr == packet)
            return false;

        return handleFunc(session, *packet);
    }

    template<typename PacketFunc, typename ... Args>
    static flatbuffers::DetachedBuffer Serialization(flatbuffers::FlatBufferBuilder& builder, PacketFunc func, Args&&... args)
    {
        {
            auto offset = func(builder, std::forward<Args>(args) ...);
            builder.Finish(offset);
            return builder.Release();
        }
    }

    static std::shared_ptr<GameServerEngine::PacketBuffer> MakePacketBuffer(const uint16 packetType, const flatbuffers::DetachedBuffer& packetData);

protected:
    std::array<PacketHandlerFunc, std::numeric_limits<uint16>::max() + 1> m_packetHandlerFuncs;
};
```

3) PacketHandler.h

```
table SC_LOCAL_PLAYER_PACKET {
    player_id: uint64;
    pos_info: FB_STRUCTS.PosInfo;
    team_type: FB_ENUMS.TEAM_TYPE;
    max_hp: uint;
    current_hp: uint;
    max_stamina: uint;
    current_stamina: uint;
    stance_type: FB_ENUMS.GENERAL_STANCE_TYPE;
}

table SC_ADD_OBJ_PACKET {
    obj_id: uint64;
    obj_type: FB_ENUMS.GAME_OBJECT_TYPE;
    team_type : FB_ENUMS.TEAM_TYPE;
    pos_info: FB_STRUCTS.PosInfo;
    max_hp: uint;
    current_hp: uint;
    max_stamina: uint;
    current_stamina: uint;
    stance_type: FB_ENUMS.GENERAL_STANCE_TYPE;
}

table SC_REMOVE_OBJ_PACKET {
    obj_id: uint64;
}

table CS_MOVE_PACKET {
    pos_info: FB_STRUCTS.PosInfo;
    move_dir: FB_ENUMS.MOVE_DIRECTION_TYPE;
}

table SC_MOVE_PACKET {
    obj_id: uint64;
    pos_info: FB_STRUCTS.PosInfo;
    sub_state: uint8;
    move_dir: FB_ENUMS.MOVE_DIRECTION_TYPE;
}

table CS_CHANGE_GENERAL_STANCE_PACKET {
}

table SC_CHANGE_GENERAL_STANCE_PACKET {
    obj_id : uint64;
    stance_type: FB_ENUMS.GENERAL_STANCE_TYPE;
    camera_target_id: uint64;
}
```

3) Tables.fbs

3) FlatBuffers 라이브러리를 사용한 패킷 프로토콜 및 핸들러 제작

FlatBuffers 스키마 기반으로 게임 오브젝트, 이동, 전투, 채팅 및 서버 간 통신 프로토콜을 설계 및 패킷 핸들러를 구현했습니다. 패킷 타입 기반 테이블을 구성해 수신 패킷을 각 함수로 분배했습니다. 또한, 네트워크로 들어오는 데이터는 바로 신뢰할 수 없기 때문에 바로 접근하지 않고 라이브러리에서 제공하는 Verifier기능을 활용해 검증에 실패하면 해당 세션의 연결을 종료하도록 구현했습니다.

2) Hybrid FSM-BT 기반 장수 AI 구현

AI 장수의 판단과 행동을 FSM의 하위 노드 단위로 설계했습니다. 공통 실행 구조와 전투/이동/점령 로직을 분리하여 코드 재사용성과 확장성을 높였습니다.

EisenValor – Implementation details

주요 구현 내용

```
class NavAgent : public Component {
public:
    explicit NavAgent(NavSystem* const navSystem);
    virtual ~NavAgent() = default;

public:
    bool Init(const dtCrowdAgentParams& params);
    virtual void Update(const float dt) override final;

public:
    void SetPlayerMode(bool isPlayer) { m_isPlayerMode = isPlayer; }
    void SetDestPos(const Vec3& destPos);
    int32 GetAgentIdx() const { return m_agentIdx; }
    void Teleport(const Vec3& destPos);

    void StopMove();
    void Remove();

    void SyncPosition(const Vec3& newPos, const Vec3& prevPos, float dt);
    void SetAsStaticObstacle();

    void SetMaxSpeed(const float maxSpeed);
    void SetMaxAcceleration(const float maxAcceleration);
    float GetMaxSpeed() const { return m_params.maxSpeed; }
    float GetMaxAcceleration() const { return m_params.maxAcceleration; }

private:
    NavSystem*      m_navSystem;
    int32           m_agentIdx;
    dtCrowdAgentParams m_params;

    Vec3           m_destPos;
    bool           m_hasTarget;

    bool           m_isPlayerMode;
};
```

4) NavAgent.h

```
auto const navQuery = m_navSystem.GetNavMeshQuery();
if(!navQuery) return;

dtQueryFilter filter;
const float extents[3] = { 0.5f, 2.5f, 0.5f }; // 계단 높이에 맞게 조정

float      newPosArr[3] = { newPos.x, newPos.y, newPos.z };
dtPolyRef  newPoly = 0;
float newNearestPt[3];

navQuery->findNearestPoly(newPosArr, extents, &filter, &newPoly, newNearestPt);

if(newPoly == 0) {
    SendPositionCorrection(clientSession, playerID, prevPos, transform.GetRotationDegree());
    return;
}

const Vec3 snapPos{ newNearestPt[0], newNearestPt[1], newNearestPt[2] };

player->SetPosition(snapPos);
player->SetRotation(transform.GetRotationDegree());
player->SetMoveDir(moveDir);

auto navAgent = player->GetComponent<NavAgent>();
if(navAgent) {
    navAgent->SyncPosition(snapPos, prevPos, m_lastDT);
}
```

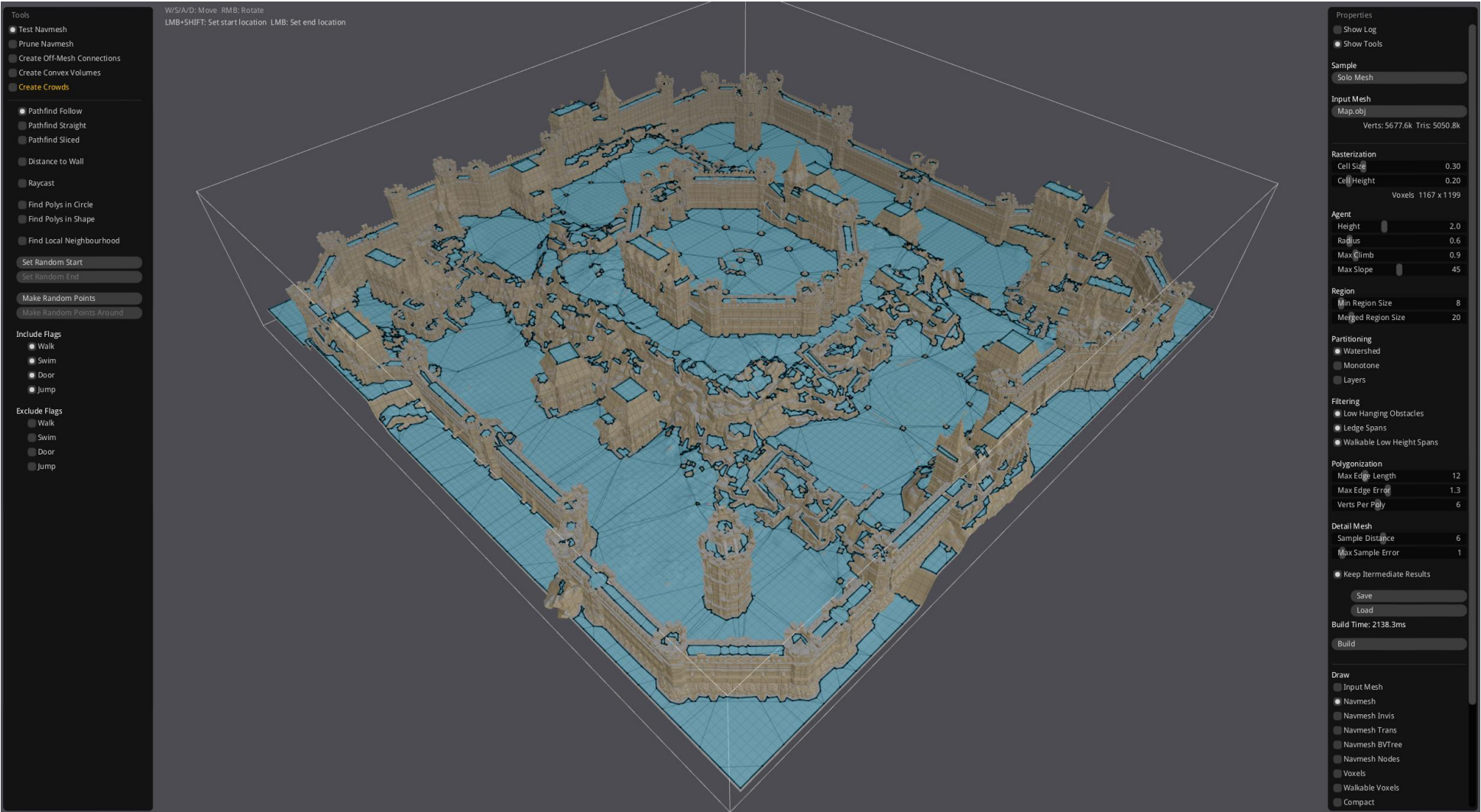
4) ClientPacketHandler.h

```
void GameServer::Contents::GameWorld::Update(const float dt)
{
    m_lastDT = m_dt;
    m_dt = dt;
    m_accDT += dt;

    uint32 loopCount{};
    while(m_accDT >= m_fixedUpdateTick && loopCount < m_maxUpdateStep) {
        if(IsGameFinish()) break;
        m_accDT -= m_fixedUpdateTick;
        ProcessEvents();
        m_navSystem.Update(m_fixedUpdateTick);
        UpdateGameWorldObjects();
        CheckCollision();
        CheckGameTime(m_fixedUpdateTick);
        CheckGameFinish();
        loopCount++;
    }

    if(m_accDT > m_fixedUpdateTick * m_maxUpdateStep) {
        m_accDT = 0.0f;
    }
}
```

4) GameWorld.cpp



4) RecastDemo 실행 후 Map.obj을 불러옴

4) RecastDetour 라이브러리를 사용한 Navigation Mesh 추출 및 NavAgent 구현

RecastDemo 프로그램을 사용하여 게임 맵의 obj파일을 불러들인 후 Navigation Mesh를 추출하였습니다. 생성된 Navigation Mesh는 경사도, 장애물, 이동 가능한 높이와 반경 등의 설정값들을 기준으로 구성 되며, 게임서버에서는 이를 활용하여 캐릭터의 이동 가능 여부와 이동 경로를 판단합니다. 또한 라이브러리에서 제공하는 경로 탐색 및 군중 이동 기능을 기반으로 NavAgent 컴포넌트를 제작하였습니다. NavAgent는 목적지까지의 이동 경로를 탐색하고 Navigation Mesh 위에서 이동 방향과 위치를 지속적으로 갱신하는 역할을 담당합니다.

EisenValor – Implementation details

주요 구현 내용

```
class DBConnectionPool : public Singleton<DBConnectionPool> {
    SINGLETON(DBConnectionPool)
public:
    bool Connect(int32 connectionCount, const WCHAR* connectionString);
    void Clear();

    DBConnection* Pop();
    void Push(DBConnection* connection);

private:
    std::mutex m_mutex;
    SQLHENV m_environment = SQL_NULL_HANDLE;
    std::vector<DBConnection*> m_connections;
};
```

5) DBConnectionPool.h

```
bool DBConnectionPool::Connect(int32 connectionCount, const WCHAR* connectionString)
{
    std::lock_guard<std::mutex> lock(m_mutex);

    if(::SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &m_environment) != SQL_SUCCESS)
        return false;

    if(::SQLSetEnvAttr(m_environment, SQL_ATTR_ODBC_VERSION, reinterpret_cast<SQLPOINTER>(SQL_OV_ODBC3), 0) != SQL_SUCCESS)
        return false;

    for(int32 i = 0; i < connectionCount; i++) {
        DBConnection* connection = new DBConnection();
        if(connection->Connect(m_environment, connectionString) == false)
            return false;

        m_connections.push_back(connection);
    }

    return true;
}

void DBConnectionPool::Clear()
{
    std::lock_guard<std::mutex> lock(m_mutex);

    if(m_environment != SQL_NULL_HANDLE) {
        ::SQLFreeHandle(SQL_HANDLE_ENV, m_environment);
        m_environment = SQL_NULL_HANDLE;
    }

    for(DBConnection* connection : m_connections)
        delete connection;

    m_connections.clear();
}

DBConnection* DBConnectionPool::Pop()
{
    std::lock_guard<std::mutex> lock(m_mutex);

    if(m_connections.empty())
        return nullptr;

    DBConnection* connection = m_connections.back();
    m_connections.pop_back();
    return connection;
}

void DBConnectionPool::Push(DBConnection* connection)
{
    std::lock_guard<std::mutex> lock(m_mutex);
    m_connections.push_back(connection);
}
```

5) DBConnectionPool.cpp

	id	varchar(50)	☐
	pw	varchar(50)	☐
	nickName	varchar(50)	☐
	winCount	int	☐
	loseCount	int	☐
	created_at	datetime2(7)	☐
			☐

5) dbo.userInfo

	열 이름	데이터 형식	Null 허용
	user_id	int	☐
	account_id	varchar(64)	☐
	nickname	varchar(64)	☐
	state	varchar(32)	☐
	room_id	int	☐
	world_id	int	☐
	transfer_expires_at	datetime2(7)	☒
	updated_at	datetime2(7)	☐
	game_server_ip	varchar(64)	☐
	game_server_port	int	☐

5) dbo.userSessionState

```
DBBind<2, 3> dbBind{*dbConnection, L"SELECT nickName, winCount, loseCount FROM dbo.userInfo WHERE id = ? AND pw = ?"};
dbBind.BindParam(0, loginID);
dbBind.BindParam(1, loginPW);
dbBind.BindCol(0, nickNameBuffer, size32(nickNameBuffer));
dbBind.BindCol(1, winCount);
dbBind.BindCol(2, loseCount);

if(dbBind.Execute() == false) {
    auto pb = LobbyServer::Make_LC_LOGIN_FAIL_PACKET("DB query failed");
    clientSession->Send(std::move(pb));
    return false;
}

if(dbBind.Fetch() == false) {
    auto pb = LobbyServer::Make_LC_LOGIN_FAIL_PACKET("Invalid id or password");
    clientSession->Send(std::move(pb));
    return true;
}
```

5) DBBind 사용예시

5) 로비 서버의 DB 연결 및 유저 데이터 관리

로비 서버에서 로그인, 회원가입과 같은 기능을 처리하기 위해

ODBC 기반의 DB 연동 구조를 직접 구현했습니다.

반복되는 DB 처리 코드를 줄이기 위해 DBConnection에서 쿼리

실행을 담당하도록 했으며 DBBind를 통해 쿼리 파라미터와 조회

결과를 쉽게 연결할 수 있도록 구성했습니다. 또한 실행 전에

필요한 값이 모두 바인딩되었는지 확인하여 바인딩 누락으로 인한

오류를 빠르게 발견할 수 있도록 구현했습니다.

여러 워커 스레드가 DB를 사용할 수 있도록 스레드 수에 맞춰

Connection을 미리 생성하고 DBConnectionPool에서 필요한

Connection을 대여하고 반납하는 방식으로 관리했습니다.

DBConnectionGuard에는 RAII 패턴을 적용하여 처리 도중

함수가 종료되더라도 사용한 Connection이 자동으로 Pool에

들어가도록 구현했습니다.

이 구조를 로그인과 회원가입, ID/닉네임 중복 검사, 게임 승패

기록에 활용했습니다. 더불어 사용자가 로비와 게임 서버 사이를

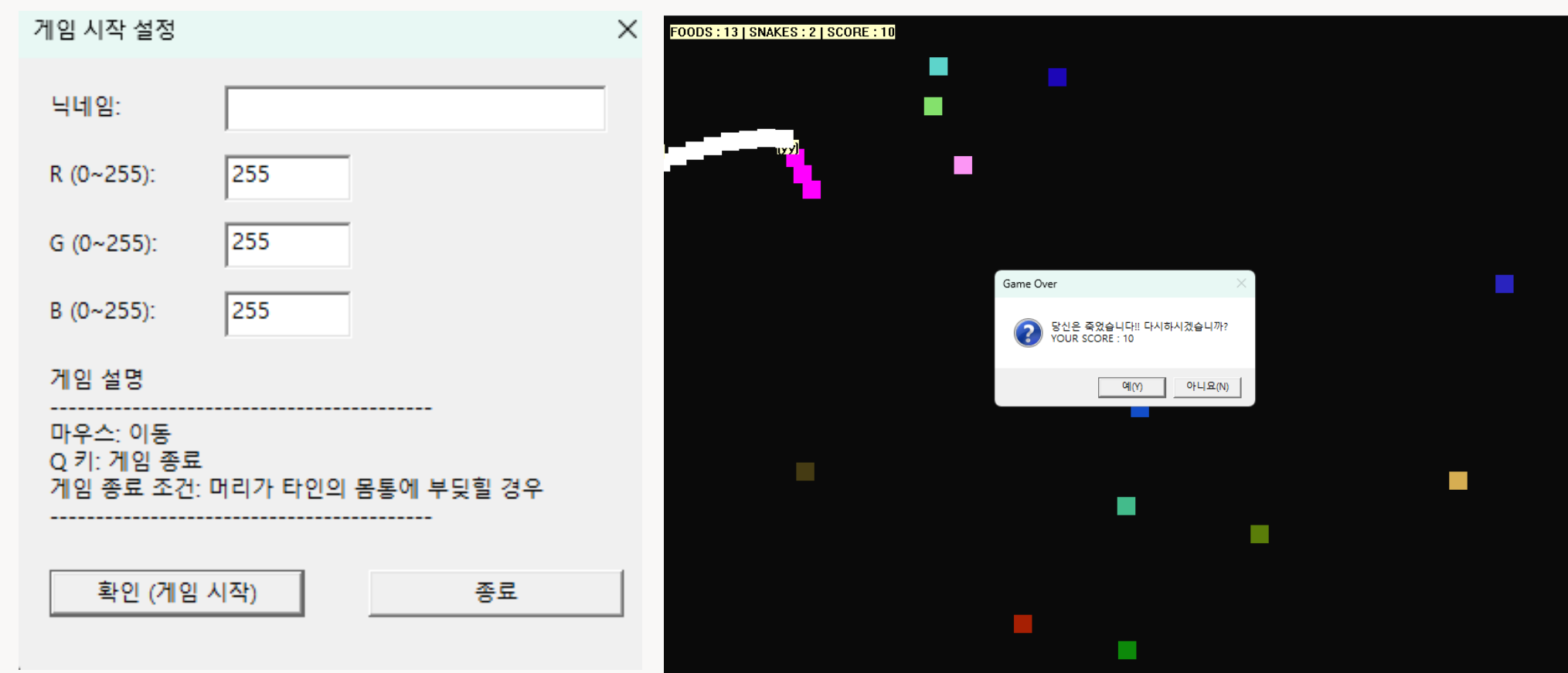
이동하거나 재접속하는 상황에 대응할 수 있도록 현재 접속 상태,

반 번호, 게임 서버 정보와 이동 유효 시간을 DB에 저장하고

복구하도록 구현했습니다.

Project 02. Snake Game

2025학년도 2학기 네트워크 게임 프로그래밍 과목 팀 텀프로젝트



TCP/IP 기반의 실시간 멀티 플레이 게임 서버를 구현하였습니다.

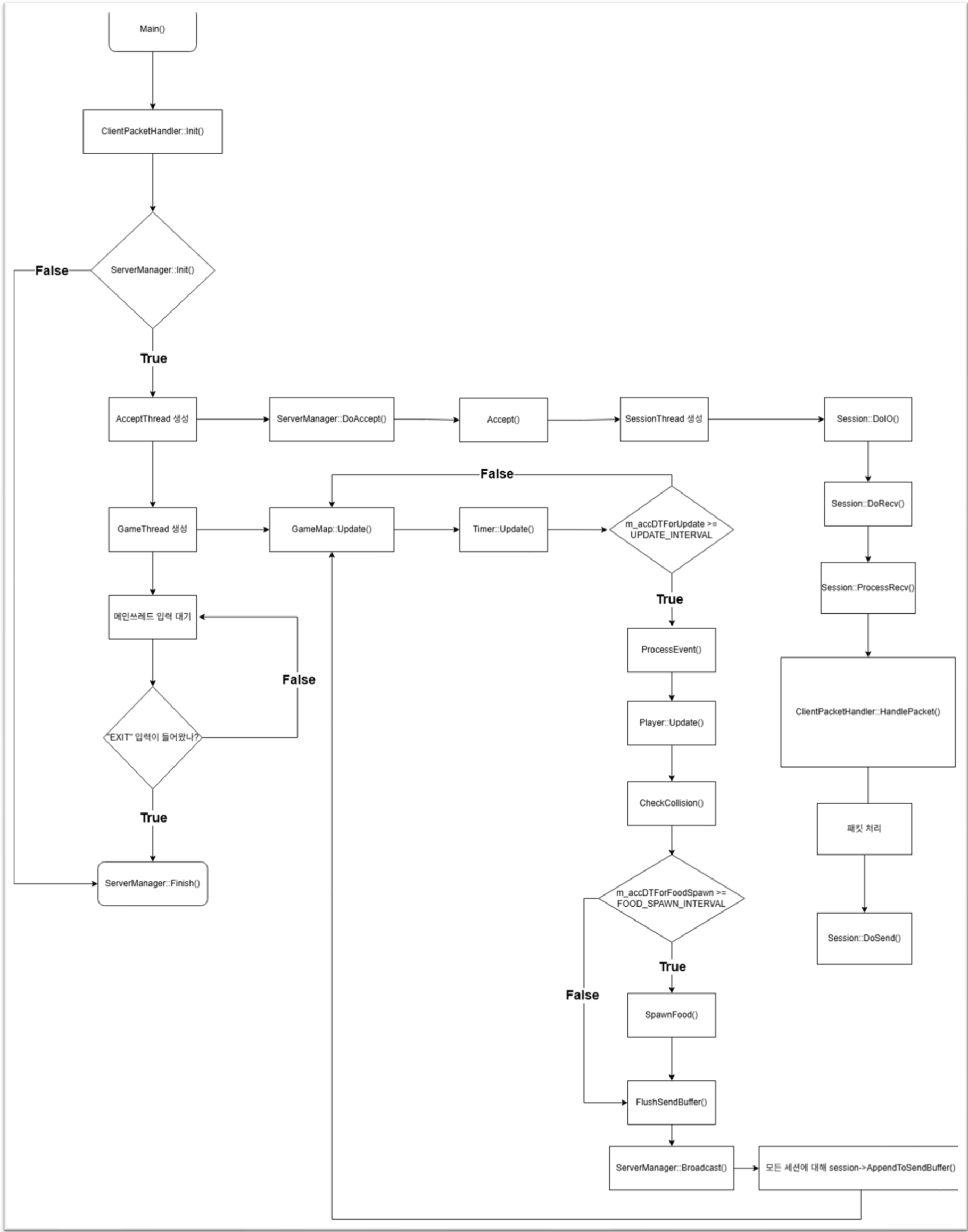
게임 도중 입/퇴장 및 플레이어의 이동을 실시간으로 동기화하였고 모든 게임 콘텐츠 관련 부분을 구현하였습니다.

게임에 필요한 모든 프로토콜을 설계하고 구현하였습니다.

게임 장르	2D 온라인 생존
개발 기간	2025.10.31 ~ 2025.12.02
개발 인원	클라이언트 프로그래머 2인, 서버 프로그래머 1인
역할 및 구현 내용	서버 프로그래머 • OOP기반 서버 프레임워크 제작 • TCP/IP를 이용한 멀티플레이 서버 구현 • 게임 도중 입/퇴장 및 이동 동기화 • 먹이 생성/삭제, 뱀 충돌처리, 점수 획득 등 모든 게임 콘텐츠 구현 • 프로토콜 설계 및 구현
개발 언어	C++20
개발 도구	Visual Studio 2022, Git/Github
Github 주소	https://github.com/yungaeng/SnakeGame

Snake Game – Server Architecture

서버 구조 설계



C++ Winsock 기반의 MultiThread 게임 서버로 네트워크 처리와 게임 로직 처리의 역할을 분리해 구성했습니다.

Main Thread는 각종 초기화 및 서버 종료 명령을 기다리고 Accept Thread는 클라이언트 접속만 전담합니다. 클라이언트가 접속하면 각 클라이언트마다 Session Thread를 생성해 패킷 송수신을 처리하며 수신된 패킷은 ClientPacketHandler를 통해 로그인, 이동, 재시작 등의 로직으로 분기됩니다.

게임 월드 상태는 별도의 Game Thread에서 관리됩니다. GameMap은 60FPS 기준으로 플레이어 위치 갱신, 몸통 이동, 충돌 판정, 먹이 생성 및 삭제를 처리합니다. Session Thread에서 직접 게임 오브젝트를 수정하지 않고 이벤트 큐를 통해 Game Thread에 작업을 전달하도록 설계하여 네트워크 I/O와 게임 월드 업데이트의 역할을 분리했습니다.

서버에서 발생한 게임 상태 변화는 SendBuffer에 누적한 뒤 ServerManager의 Broadcast를 통해 모든 클라이언트에게 전송됩니다. 이를 통해 플레이어 이동, 먹이 생성, 충돌, 사망과 같은 상태 변화가 각 클라이언트에 동기화되도록 구현했습니다.

Snake Game – Implementation details

주요 구현 내용

```
int main()
{
    ClientPacketHandler::Init();

    try {
        if(false == MANAGER(ServerManager)->Init())
            throw std::runtime_error("Failed IOManager Init!");

        std::jthread acceptThread{ [](const std::stop_token& st) { MANAGER(ServerManager)->DoAccept(st); } };
        std::jthread gameThread{ [](const std::stop_token& st) { MANAGER(GameMap)->Update(st); } };

        // 메인쓰레드
        std::string word;

        while(true) {
            std::cin >> word;
            if("EXIT" == word) {
                MANAGER(ServerManager)->Finish();
                break;
            }
        }
    }
    catch(const std::exception& e) {
        std::cout << e.what() << std::endl;
    }
}
```

1) main.cpp

```
auto session = std::make_shared<Session>(clientSocket);
const uint64 id = session->GetID();

{
    std::lock_guard<std::mutex> lk{ m_sessionMutex };
    m_sessions.try_emplace(id, session);
}

{
    std::lock_guard<std::mutex> lk{ m_sessionThreadsMutex };
    m_sessionThreads.emplace_back([this, session](const std::stop_token& st) { session->DoIO(st); });
}
```

1) ServerManager.cpp

```
bool Process_C2S_LOGIN_PACKET(const std::shared_ptr<Session>& session, const C2S_LOGIN_PACKET& recvPkt)
{
    // Session Thread가 수행중

    const bool hasName = MANAGER(GameMap)->FindPlayerName(recvPkt.name);

    // 만약, 로그인 이름이 이미 게임에 있으면 SC_LOGIN 발송
    if(hasName) {
        S2C_LOGIN_FAIL_PACKET sendPkt;
        session->AppendPkt(sendPkt);
    }
    // 로그인이 정상적으로 되었다면 게임맵에 플레이어 추가
    else {

        session->SetSessionInfo(SessionInfo{ recvPkt.name, recvPkt.color });
        auto player = std::make_shared<Player>();

        const Pos pos{ GetRandomPos() };
        S2C_LOGIN_OK_PACKET sendPkt;
        sendPkt.id = player->GetID();
        sendPkt.x = pos.x;
        sendPkt.y = pos.y;
        session->AppendPkt(sendPkt);

        player->SetName(recvPkt.name);
        player->SetColor(recvPkt.color);
        player->SetPos(pos);
        player->SetSession(session);
        session->SetPlayer(player);

        MANAGER(GameMap)->AddEvent([p = std::move(player)](){
            {
                MANAGER(GameMap)->AddGameObject(std::move(p));
            }
        });

        return true;
    }
}
```

2) ClientPacketHandler.cpp

```
void GameMap::Update(const std::stop_token& st)
{
    while(false == st.stop_requested()) {
        m_timer.Update();
        const float dt = m_timer.GetDT();
        m_accDTForUpdate += dt;
        m_accDTForFoodSpawn += dt;

        if(m_accDTForUpdate ≥ UPDATE_INVERVAL) {
            ProcessEvent();

            for(const auto& [playerID, player] : m_players) {
                if(false == player->IsAlive()) continue;
                player->Update(m_accDTForUpdate);
            }

            CheckCollision();
            if(m_accDTForFoodSpawn ≥ FOOD_SPAWN_INTERVAL) {
                SpawnFood();
                m_accDTForFoodSpawn = 0.f;
            }

            FlushSendBuffer();
            m_accDTForUpdate = 0.f;
        }
    }

    std::cout << "Finish GameThread!" << std::endl;
}
```

3) GameMap.cpp

2) 이벤트 큐를 통한 쓰레드 간 작업 전달

클라이언트 패킷으로 인해 발생하는 게임 상태 변경은 이벤트 큐에 등록하고, 실제 오브젝트 추가/삭제는 Game Thread에서 처리하도록 설계했습니다. 이를 통해 여러 Session Thread가 게임 월드 상태를 수정하는 상황을 없앴습니다.

3) Game Thread 기반 게임 상태 관리

게임 상태는 Game Thread에서 관리되고 있으며 60FPS 기준으로 플레이어 이동, 몸통 위치 보정, 먹이 생성, 충돌 판정을 처리합니다.

1) 멀티 쓰레드 서버 구조

Accept 처리, 네트워크 I/O, 게임 월드 업데이트를 각각 별도 쓰레드로 분리하였습니다. 이를 통해 네트워크 I/O가 게임 로직 업데이트 주기에 직접 영향을 주지 않도록 설계했습니다. 쓰레드 간 동기화에는 Mutex 객체를 사용하였습니다.

Snake Game – Protocol Architecture

프로토콜 구조 설계

```
bool Process_HANDLE_INVALID_PACKET(const std::shared_ptr<Session>&, const char* const);

bool Process_C2S_LOGIN_PACKET(const std::shared_ptr<Session>&, const C2S_LOGIN_PACKET&);
bool Process_C2S_RESTART_PACKET(const std::shared_ptr<Session>&, const C2S_RESTART_PACKET&);
bool Process_C2S_MOVE_PACKET(const std::shared_ptr<Session>&, const C2S_MOVE_PACKET&);

using PacketHandlerFunc = bool(*)(const std::shared_ptr<Session>&, const char* const);
extern inline constinit std::array<PacketHandlerFunc, std::numeric_limits<uint16>::max() + 1> PacketHandlerFuncs{};

class ClientPacketHandler {
private:
    ClientPacketHandler() = delete;
    ~ClientPacketHandler() = delete;
    ClientPacketHandler(const ClientPacketHandler&) = delete;
    ClientPacketHandler& operator= (const ClientPacketHandler&) = delete;
    ClientPacketHandler(ClientPacketHandler&) noexcept = delete;
    ClientPacketHandler& operator= (ClientPacketHandler&) noexcept = delete;

public:
    template<typename PacketType, typename HandleFunc>
    static bool HandlePacket(HandleFunc func, const std::shared_ptr<Session>& session, const char* const buffer) noexcept
    {
        return func(session, *reinterpret_cast<const PacketType*>(buffer));
    }

    static void Init() noexcept
    {
        for(auto& func : PacketHandlerFuncs) {
            func = Process_HANDLE_INVALID_PACKET;

            PacketHandlerFuncs[static_cast<uint8>(PACKET_ID::C2S_LOGIN)] = [](const std::shared_ptr<Session>& session, const char* const buffer)
            -> bool { return HandlePacket<C2S_LOGIN_PACKET>(Process_C2S_LOGIN_PACKET, session, buffer); };
            PacketHandlerFuncs[static_cast<uint8>(PACKET_ID::C2S_RESTART)] = [](const std::shared_ptr<Session>& session, const char* const buffer)
            -> bool { return HandlePacket<C2S_RESTART_PACKET>(Process_C2S_RESTART_PACKET, session, buffer); };
            PacketHandlerFuncs[static_cast<uint8>(PACKET_ID::C2S_MOVE)] = [](const std::shared_ptr<Session>& session, const char* const buffer)
            -> bool { return HandlePacket<C2S_MOVE_PACKET>(Process_C2S_MOVE_PACKET, session, buffer); };
        }

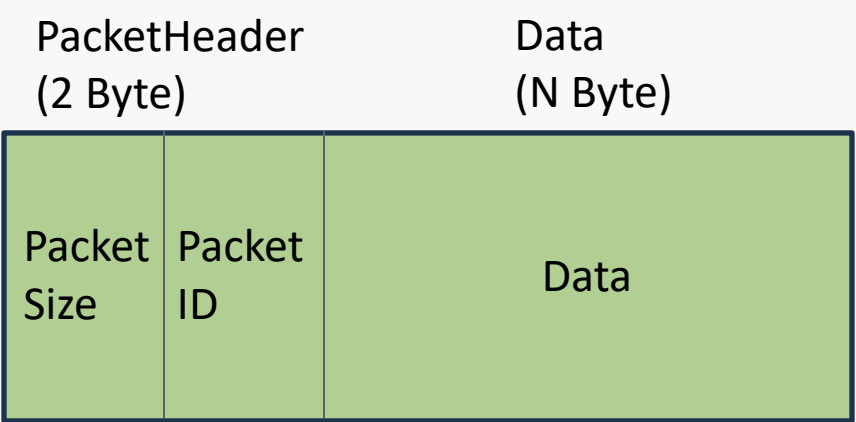
        static inline bool HandlePacket(const std::shared_ptr<Session>& session, const char* const buffer)
        {
            const PacketHeader* const packetHeader = reinterpret_cast<const PacketHeader*>(buffer);
            return std::invoke(PacketHandlerFuncs[static_cast<uint8>(packetHeader->packetID)], session, buffer);
        }
    }
};
```

1) ClientPacketHandler.h

1)패킷 처리 함수 매핑

함수 포인터 테이블을 활용해 O(1)의 시간복잡도로 패킷을
찾도록 구현하여 각 패킷을 적절한 처리 함수로
매핑하였습니다.

새로운 패킷은 핸들러 등록만으로 패킷을 처리할 수 있는
환경을 만들었습니다.



```
#pragma pack(push, 1)
struct PacketHeader {
    uint8 packetSize;
    uint8 packetID;
};

enum class PACKET_ID : uint8 {
    //client
    C2S_LOGIN, C2S_MOVE, C2S_RESTART, C2S_LEAVE,
    //server
    S2C_LOGIN_OK, S2C_LOGIN_FAIL, S2C_SNAKE, S2C_FOOD, S2C_MOVE, S2C_DEL_SNAKE, S2C_DEL_FOOD, S2C_EAT_FOOD, S2C_SNAKE_MOVE,
};

//client

/// <summary>
/// LOGIN
/// </summary>
struct C2S_LOGIN_PACKET : public PacketHeader {
    wchar_t name[10];
    COLORREF color;
    C2S_LOGIN_PACKET() : PacketHeader{ sizeof(C2S_LOGIN_PACKET), static_cast<uint8>(PACKET_ID::C2S_LOGIN) } {}
};

/// <summary>
/// MOVE
/// </summary>
struct C2S_MOVE_PACKET : public PacketHeader {
    float x, y;
    C2S_MOVE_PACKET() : PacketHeader{ sizeof(C2S_MOVE_PACKET), static_cast<uint8>(PACKET_ID::C2S_MOVE) } {}
};
```

2) protocol.h

2) Structure Format Packet

패킷 헤더에 사이즈와 ID를 포함하였고 모든 패킷이
이를 상속하도록 설계했습니다. 패킷은 관리하기 쉬운
Structure Format으로 만들었습니다.

3) 패킷 처리

수신 시, 정상적인 데이터만 처리하도록 패킷 헤더를
먼저 검사하여 패킷의 완전성을 검증한 후에 패킷을
처리 했습니다. 이를 통해 TCP에서 발생하는 패킷
분할 상황을 안정적으로 처리할 수 있습니다.

```
uint32 Session::ProcessRecv(const char* const readPos, const uint32 dataSize)
{
    uint32 processedDataLen{};

    while(true) {
        const uint32 dataLen = dataSize - processedDataLen;

        if(dataLen <= sizeof(PacketHeader)) break;

        const PacketHeader* const header = reinterpret_cast<const PacketHeader*>(&readPos[processedDataLen]);

        if(dataLen < header->packetSize) break;

        OnRecvPacket(&readPos[processedDataLen]);

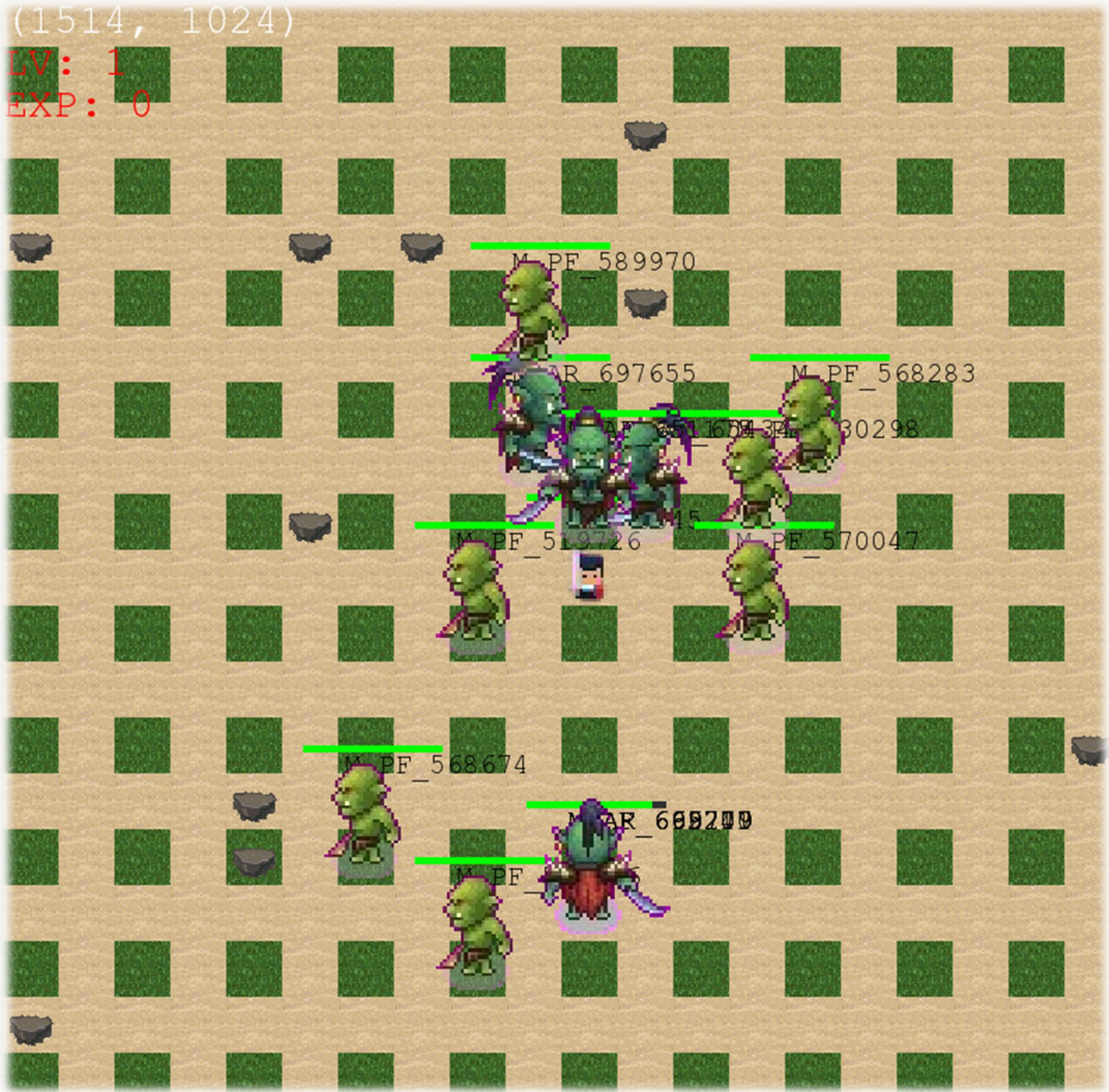
        processedDataLen += dataLen;
    }

    return processedDataLen;
}
```

3) Session.cpp

Project 03. SIMPLEST MMORPG

2025학년도 1학기 게임 서버 프로그래밍 과목 개인 텀프로젝트



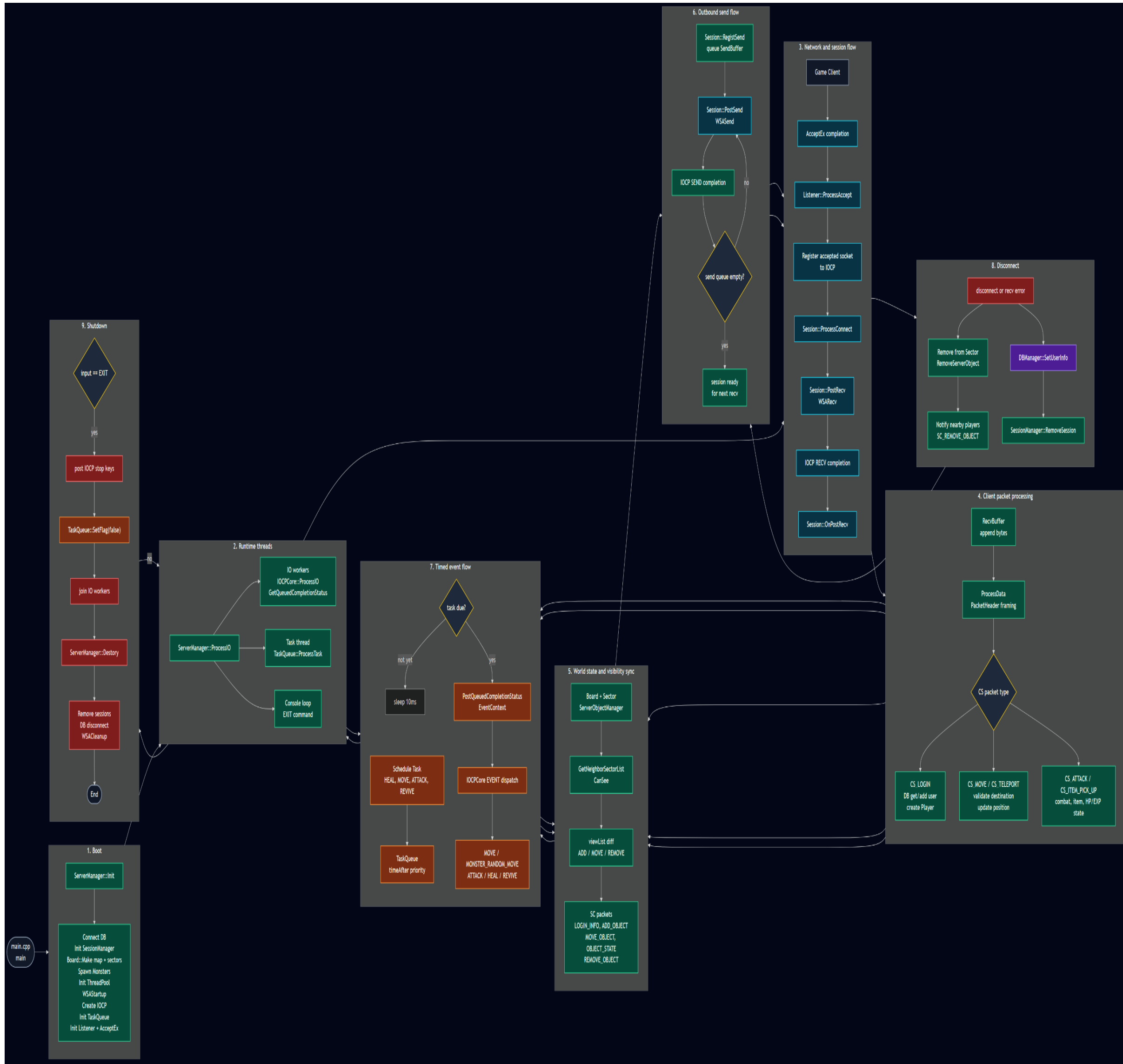
IOCP를 이용해 멀티 쓰레드 MMORPG 게임 서버를 구현하였습니다.

로그인, 캐릭터 이동, 전투, 시야처리와 공간분할, NPC 길 찾기, DB 연동을 해보면서 MMORPG 서버에 대한 기본적인 구조를 학습했습니다.

게임 장르	탐류 2D MMORPG
개발 기간	2025.05.21 ~ 2025.06.15
개발 인원	1인
역할 및 구현 내용	• SFML을 이용한 2D 게임 클라이언트 제작 • OOP 기반 서버 프레임워크 및 IOCP 멀티 쓰레드 서버 구현 • 프로토콜 설계 및 구현 • 2000 X 2000 크기의 맵에서 시야처리 및 공간분할 구현 • 플레이어 이동동기화 및 전투 구현 • A* 알고리즘을 이용한 몬스터 길 찾기 구현 • 드랍 아이템 및 장애물 구현 • 더미 클라이언트를 통한 동접 테스트 • DB를 통한 유저 정보 관리
개발 언어	C++20
개발 도구 & 라이브러리	Visual Studio 2022, SSMS 21, Git/Github, Intel TBB
Github 주소	https://github.com/syyundev/SIMPLEST-MMORPG-SERVER

SIMPLEST MMORPG – Server Architecture

서버 구조 설계



1) IOCP 기반 멀티 쓰레드 서버

Windows IOCP를 사용해 다중 클라이언트 접속을 처리하는 멀티쓰레드 서버 구조를 구현했습니다. Listener는 클라이언트 연결을 수락하고 Session은 recv/send context를 관리하며, IOCPCore는 완료 이벤트를 여러 Worker Thread에서 Dispatch하도록 구성했습니다. 쓰레드 간 동기화에는 Mutex 객체를 사용하였습니다.

2) Sector 기반 월드 동기화

전체 월드를 Sector 단위로 분할하여 플레이어 주변 객체만 탐색하도록 구현했습니다. 이동 전후의 viewList를 비교해 새로 보이는 객체는 ADD, 이동 중인 객체는 MOVE, 시야에서 벗어난 객체는 REMOVE 패킷으로 동기화했습니다.

3) 서버 기반 게임 로직

클라이언트는 이동/공격 요청만 보내고 이동 가능 여부, 장애물 체크, 공격 판정, HP/EXP/Level 처리 판정은 서버가 하도록 구현하였습니다.

4) DB 연동

ODBC 기반 유저 정보 저장/로드 및 로그아웃/접속 종료 시 위치, HP, EXP, Level을 저장합니다.

SIMPLEST MMORPG – Implementation details

주요 구현 내용

```
std::unordered_set<int> Board::GetNeighborSectorList(const Pos pos, const int viewRange)
{
    unordered_set<int> neighborSecList;

#ifdef ENABLE_SPACE_DEVISION
    const int minViewX( std::max(0, pos.x - viewRange) );
    const int maxViewX( std::min(W_WIDTH - 1, pos.x + viewRange) );
    const int minViewY( std::max(0, pos.y - viewRange) );
    const int maxViewY( std::min(W_HEIGHT - 1, pos.y + viewRange) );

    int minSecX = minViewX / Board::SECTOR_SIZE;
    int maxSecX = maxViewX / Board::SECTOR_SIZE;
    int minSecY = minViewY / Board::SECTOR_SIZE;
    int maxSecY = maxViewY / Board::SECTOR_SIZE;

    minSecX = std::max(0, minSecX);
    maxSecX = std::min(Board::SECTOR_X_COUNT - 1, maxSecX);
    minSecY = std::max(0, minSecY);
    maxSecY = std::min(Board::SECTOR_Y_COUNT - 1, maxSecY);

    for(int sy = minSecY; sy <= maxSecY; ++sy) {
        for(int sx = minSecX; sx <= maxSecX; ++sx) {
            const int secID = mSectors[sy][sx]->GetID();
            if(neighborSecList.find(secID) == neighborSecList.end()) {
                neighborSecList.insert(secID);
            }
        }
    }
#else
    (void)pos;
    (void)viewRange;
    for(const auto& entry : m_hash) {
        neighborSecList.insert(entry.first);
    }
#endif

    return neighborSecList;
}

bool Board::CanSee(const Pos from, const Pos to, const int viewRange)
{
#ifdef ENABLE_VIEW_PROCESSING
    if(abs(from.x - to.x) > viewRange) return false;
    return abs(from.y - to.y) <= viewRange;
#else
    (void)from;
    (void)to;
    (void)viewRange;
    return true;
#endif
}
```

1) Board.cpp

```
std::vector<Pos> Monster::DoAstar()
{
    auto player = m_target.lock();

    if(nullptr == player)
        return();

    const Pos startPos = GetPos();
    const Pos destPos = player->GetPos();

    static const Pos dirs[4] = { Pos(1,0), Pos(-1,0), Pos(0,1), Pos(0,-1) };

    // fScore가 작은 노드가 우선순위 최상
    std::priority_queue<Node, std::vector<Node>, std::greater<Node>> openSet;
    std::unordered_map<Pos, int, PosHash> gScore; // 각 Pos 까지의 최단 이동 비용 기록
    std::unordered_map<Pos, Pos, PosHash> cameFrom; // 경로 역추적, Pos 도달 시 바로 이전 Pos 저장함.

    gScore[startPos] = 0;
    openSet.push({ startPos, 0, heuristic(startPos, destPos) });

    while(!openSet.empty()) {
        Node current = openSet.top();
        openSet.pop();

        // 현재 위치가 도착점이라면, 출발점부터 목적지까지 경로 백터 재구성
        if(current.pos == destPos)
            return ReconstructPath(cameFrom, current.pos);

        for(const Pos& d : dirs) {
            Pos neighbor{ static_cast<short>(current.pos.x + d.x),
                          static_cast<short>(current.pos.y + d.y) };
            if(neighbor.x < 0 || neighbor.x >= Board::BOARD_WIDTH
               || neighbor.y < 0 || neighbor.y > Board::BOARD_HEIGHT)
                continue;
            if(MANAGER(Board)->m_boards[neighbor.y][neighbor.x] == TILE_TYPE::OBSTACLE)
                continue;

            const int tentativeG = current.g + 1;
            if(!gScore.count(neighbor) || tentativeG < gScore[neighbor]) {
                gScore[neighbor] = tentativeG;
                const int fScore = tentativeG + heuristic(neighbor, destPos);
                openSet.push({ neighbor, tentativeG, fScore });
                cameFrom[neighbor] = current.pos;
            }
        }
    }

    return();
}
```

2) Monster.cpp

```
void TaskQueue::ProcessTask() noexcept
{
    do {
        do {
            Task task;
            if(false == m_taskQueue.try_pop(task)) {
                break;
            }
        } else {
            if(task.timeAfter > std::chrono::high_resolution_clock::now()) {
                m_taskQueue.push(task);
                break;
            }

            switch(const auto type = task.taskType) {
                case EVENT_TYPE::MOVE:
                {
                    EventContext* context = new EventContext;
                    context->type = task.taskType;
                    context->ai_target_obj = task.targetObjID;
                    PostQueuedCompletionStatus(m_iocpHandle, 1, task.objID, context);
                    break;
                }

                case EVENT_TYPE::ATTACK:
                {
                    EventContext* context = new EventContext;
                    context->type = task.taskType;
                    context->ai_target_obj = task.targetObjID;
                    PostQueuedCompletionStatus(m_iocpHandle, 1, task.objID, context);
                    break;
                }

                case EVENT_TYPE::HEAL:
                {
                    EventContext* context = new EventContext;
                    context->type = task.taskType;
                    PostQueuedCompletionStatus(m_iocpHandle, 1, task.objID, context);
                    break;
                }

                case EVENT_TYPE::REVIVE:
                {
                    EventContext* context = new EventContext;
                    context->type = task.taskType;
                    PostQueuedCompletionStatus(m_iocpHandle, 1, task.objID, context);
                    break;
                }

                default:
                    break;
            }
        }

        } while(m_flag);
        this_thread::sleep_for(chrono::milliseconds(10));
    } while(m_flag);
}
```

3) TaskQueue.cpp

```
if(MANAGER(Board)->CanGo(nextPos)) {
    myPlayer->SetLastMoveTime(current_time);
    myPlayer->SetPos(nextPos);

    auto oldSector = MANAGER(Board)->GetSector(prevPos);
    auto newSector = MANAGER(Board)->GetSector(nextPos);

    if(oldSector != newSector) {
        const int myID( myPlayer->GetID() );
        MANAGER(Board)->GetSector(prevPos)->Remove(myID);
        MANAGER(Board)->GetSector(nextPos)->Add(myID);
    }

    unordered_set<int> nearList;

    myPlayer->m_viewLock.lock_shared();
    auto oldViewList = myPlayer->m_viewList;
    myPlayer->m_viewLock.unlock_shared();

    auto neighborSecList = MANAGER(Board)->GetNeighborSectorList(myPlayer->GetPos());

    for(const int secID : neighborSecList) {
        auto sector = MANAGER(Board)->GetSector(secID);

        auto objList = sector->GetObjList();

        for(const int objID : objList) {
            auto obj = MANAGER(ServerObjectManager)->GetGameObject(objID);

            if(obj == nullptr || obj->GetServerState() != ST_INGAME) continue;

            if(obj->GetID() == myPlayer->GetID()) continue;

            if(MANAGER(Board)->CanSee(myPlayer->GetPos(), obj->GetPos()))
                nearList.insert(obj->GetID());
        }
    }

    // 나에게 이동좌표 보내주기
    SC_MOVE_PACKET sendPkt;
    sendPkt.size = sizeof(sendPkt);
    sendPkt.type = SC_MOVE_PACKET;
    sendPkt.id = myPlayer->GetID();
    sendPkt.x = myPlayer->GetPos().x;
    sendPkt.y = myPlayer->GetPos().y;
    sendPkt.move_time = static_cast<int>(myPlayer->GetLastMoveTime());
    sendPkt.dir = myPlayer->GetDir();

    auto sendBuffer = make_shared<SendBuffer>();
    sendBuffer->Append(sendPkt);
    session->RegistSend(std::move(sendBuffer));
}

for(const int objID : nearList) {
    auto obj = MANAGER(ServerObjectManager)->GetGameObject(objID);

    if(obj == nullptr || obj->GetServerState() != ST_INGAME) continue;

    if(obj->GetID() == myPlayer->GetID()) continue;

    switch(auto type = static_cast<OBJECT_TYPE>(obj->GetObjType())) {
        case OBJECT_TYPE::PLAYER:
        {
            auto player = std::static_pointer_cast<Player>(obj);

            player->m_viewLock.lock_shared();
            if(player->m_viewList.end() != player->m_viewList.find(myPlayer->GetID())) {
                player->m_viewLock.unlock_shared();
            }
        }
    }
}
```

4) PacketFunc.cpp

1) 시야 처리와 공간 분할

2000 X 2000 크기의 맵 전체를 매번 탐색하지 않고 플레이어의 위치와 시야 범위를 기준으로 인접 섹터만 계산하도록 구현했습니다. 이를 통해 객체 탐색 범위를 줄이고 이동/전투/동기화 처리 시 불필요한 연산을 최소화했습니다.

2) A*를 이용한 몬스터 경로 탐색

몬스터가 플레이어를 추적할 수 있도록 A*알고리즘 기반 경로 탐색을 구현했습니다. 장애물과 맵 경계를 고려해 이동 가능한 경로를 계산하고 타겟과 가까워지면 공격 이벤트를 예약하는 방식으로 AI행동을 처리했습니다.

3) 타이머 이벤트 큐 / IOCP 연동

몬스터 이동, 공격, 회복, 부활처럼 일정 시간 뒤 실행되어야 하는 작업을 우선순위 큐에 등록하도록 구현했습니다. 예약 시간이 된 작업은 PQCS를 통해 IOCP에 전달하여 네트워크 I/O와 게임 이벤트를 같은 처리 흐름에서 관리했습니다.

4) 플레이어 이동 및 주변 객체 동기화

플레이어 이동 요청을 처리할 때, 위치 변경에 따라 섹터 정보를 갱신하도록 구현했습니다. 이동 후에는 기존 시야 목록과 새로 계산한 주변 객체 목록을 비교해, 클라이언트에 필요한 ADD, MOVE, REMOVE 패킷만 전송하도록 처리했습니다.

SIMPLEST MMORPG – Performance Improvement

병목분석을 통한 성능 개선 – Gather I/O 방식을 이용한 네트워크 전송 최적화

```
#else
shared_ptr<SendBuffer> sendBuffer{ nullptr };
if(false == mSendQueue.try_pop(sendBuffer)) {
    mSendContext.owner = nullptr;
    mSendRegistred.store(false);
    return;
}

mSendContext.sendBuffers.push_back(std::move(sendBuffer));

WSABUF wsaBuf;
wsaBuf.buf = const_cast<char*>(mSendContext.sendBuffers.front()->GetBuffer());
wsaBuf.len = mSendContext.sendBuffers.front()->GetDataSize();

DWORD sizeSent{};
if(SOCKET_ERROR == WSASend(m_socket, &wsaBuf, 1, &sizeSent, 0, &mSendContext, nullptr)) {
    int errorCode = ::WSAGetLastError();
    if(errorCode != WSA_IO_PENDING) {
        print_error_message(errorCode);
        mSendContext.owner = nullptr;
        mSendContext.sendBuffers.clear();
        mSendRegistred.store(false);
    }
}
}
#endif
```

```
#if USE_GATHER_IO
vector<WSABUF> wsaBufs;

while(mSendQueue.empty() == false) {
    shared_ptr<SendBuffer> sendBuffer{ nullptr };
    if(false == mSendQueue.try_pop(sendBuffer))
        continue;
    mSendContext.sendBuffers.push_back(std::move(sendBuffer));
}

for(const shared_ptr<SendBuffer>& sendBuffer : mSendContext.sendBuffers) {
    WSABUF wsaBuf;
    wsaBuf.buf = const_cast<char*>(sendBuffer->GetBuffer());
    wsaBuf.len = sendBuffer->GetDataSize();
    wsaBufs.push_back(wsaBuf);
}

DWORD sizeSent{};
if(SOCKET_ERROR == WSASend(m_socket, wsaBufs.data(), static_cast<DWORD>(wsaBufs.size()), &sizeSent, 0, &mSendContext, nullptr)) {
    int errorCode = ::WSAGetLastError();
    if(errorCode != WSA_IO_PENDING) {
        print_error_message(errorCode);
        mSendContext.owner = nullptr;
        mSendContext.sendBuffers.clear();
        mSendRegistred.store(false);
    }
}
}
```

Visual Studio의 Concurrency Visualizer를 활용해 서버 성능을 분석한 결과, 캐릭터 이동 과정에서 Send 함수가 매우 빈번하게 호출되는 것을 확인했습니다. 이동 패킷이 보낼 때마다 개별적으로 전송하면서 시스템 콜과 네트워크 I/O 요청이 반복되었고 이는 접속자 수가 증가할수록 CPU 사용량과 전송 처리 부하를 높이는 요인이 되었습니다.

이를 개선하기 위해 여러 메모리 버퍼를 하나의 I/O 요청으로 전송하는 Gather I/O 방식을 적용했습니다. 패킷 헤더와 데이터 영역을 별도의 버퍼로 유지하면서도 WSASend에 여러 개의 WSABUF를 전달하여 한 번에 전송하도록 구현했습니다.

그 결과, 패킷 조립을 위한 불필요한 메모리 복사를 줄이고 개별 패킷마다 발생하던 Send 호출 횟수와 시스템 콜 오버헤드를 감소시켰습니다. 이를 통해 이동 패킷이 집중되는 상황에서도 네트워크 전송 처리의 효율성을 올렸습니다.

결과: Send 함수의 실행 지분이 18.22% -> 8.55% 로 감소

이름	파일 샘플	전용 샘플	포함(백분율)	제외(백분율)	세부 정보
ucrtbase.dll	23,526	0	96.62%	0.00%	ucrtbase!0x2cd30
ucrtbase.dll!0x2cd30	23,526	0	96.62%	0.00%	ucrtbase!0x2cd30
server_release.exe!std::thread::Invoke<std::tuple<'Server	22,173	0	91.06%	0.00%	Server_Release!std::thread::Invoke<std::tuple
server_release.exe!IOCPCore::ProcessIO	22,173	1,209	91.06%	4.97%	Server_Release!IOCPCore::ProcessIO!IOCPCor
server_release.exe!Monster::Move	6,444	742	26.46%	3.05%	Server_Release!Monster::Move!Monster.cpp:6
server_release.exe!Session::PostSend	4,436	10	18.22%	0.04%	Server_Release!Session::PostSend!Session
kernelbase.dll!0x231ac	2,654	0	10.90%	0.00%	KernelBase!0x231ac
ntdll.dll!0x160194	2,652	79	10.89%	0.32%	ntdll!0x160194
server_release.exe!Sector::GetObjList	1,362	30	5.59%	0.12%	Server_Release!Sector::GetObjList!Sector.cpp:
server_release.exe!std::Hash<std::_Uset_traits<int,stri	1,042	232	4.28%	0.95%	Server_Release!std::Hash<std::_Uset_traits<ir
server_release.exe!Concurrency::details::_Concurrent_f	1,221	1,092	5.01%	4.48%	Server_Release!Concurrency::details::_Concur
server_release.exe!Session::OnPostRecv	1,203	2	4.94%	0.01%	Server_Release!Session::OnPostRecv!Session.1
server_release.exe!Monster::Attack	1,003	86	4.12%	0.35%	Server_Release!Monster::Attack!Monster.cpp:
server_release.exe!std::_Insert_string<char,std::char_tra	824	1	3.38%	0.00%	Server_Release!std::_Insert_string<char,std::ch
server_release.exe!std::thread::Invoke<std::tuple<'Server	1,353	0	5.56%	0.00%	Server_Release!std::thread::Invoke<std::tuple
server_release.exe!TaskQueue::ProcessTask	1,353	4	5.56%	0.02%	Server_Release!TaskQueue::ProcessTask!TaskC
ntoskrnl.exe	654	0	2.69%	0.00%	
ntoskrnl.exe!0x6bca58	557	0	2.29%	0.00%	ntoskrnl!0x6bca58

Gather I/O 전

이름	파일 샘플	전용 샘플	포함(백분율)	제외(백분율)	세부 정보
ucrtbase.dll	10,873	0	96.11%	0.00%	
ucrtbase.dll!0x2cd30	10,873	0	96.11%	0.00%	ucrtbase!0x2cd30
server_release.exe!std::thread::Invoke<std::tuple<'Server	10,131	0	89.55%	0.00%	Server_Release!std::thread::Invoke<std::tuple
server_release.exe!IOCPCore::ProcessIO	10,131	635	89.55%	5.61%	Server_Release!IOCPCore::ProcessIO!IOCPCor
server_release.exe!Monster::Move	3,248	423	28.71%	3.74%	Server_Release!Monster::Move!Monster.cpp:6
kernelbase.dll!0x231ac	1,286	0	11.37%	0.00%	KernelBase!0x231ac
server_release.exe!Session::PostSend	967	6	8.55%	0.05%	Server_Release!Session::PostSend!Session
ws2_32.dll!0x10d95	944	0	8.34%	0.00%	ws2_32!0x10d95
server_release.exe!Sector::GetObjList	736	15	6.51%	0.13%	Server_Release!Sector::GetObjList!Sector.cpp:
server_release.exe!Concurrency::details::_Concurrent_f	632	576	5.59%	5.09%	Server_Release!Concurrency::details::_Concur
server_release.exe!Session::OnPostRecv	618	1	5.46%	0.01%	Server_Release!Session::OnPostRecv!Session.1
server_release.exe!Monster::Attack	505	42	4.46%	0.37%	Server_Release!Monster::Attack!Monster.cpp:
server_release.exe!std::_Insert_string<char,std::char_tra	470	0	4.15%	0.00%	Server_Release!std::_Insert_string<char,std::ch
server_release.exe!std::thread::Invoke<std::tuple<'Server	742	0	6.56%	0.00%	Server_Release!std::thread::Invoke<std::tuple
server_release.exe!TaskQueue::ProcessTask	742	1	6.56%	0.01%	Server_Release!TaskQueue::ProcessTask!TaskC
ntoskrnl.exe	340	0	3.01%	0.00%	
ntoskrnl.exe!0x6bca58	303	0	2.68%	0.00%	ntoskrnl!0x6bca58
ntoskrnl.exe!0x97c4af	254	0	2.25%	0.00%	ntoskrnl!0x97c4af
ntoskrnl.exe!0x25e6a2	252	0	2.23%	0.00%	ntoskrnl!0x25e6a2

Gather I/O 후

※테스트 환경

CPU: AMD RYZEN 9 8940HX

RAM: 64GB

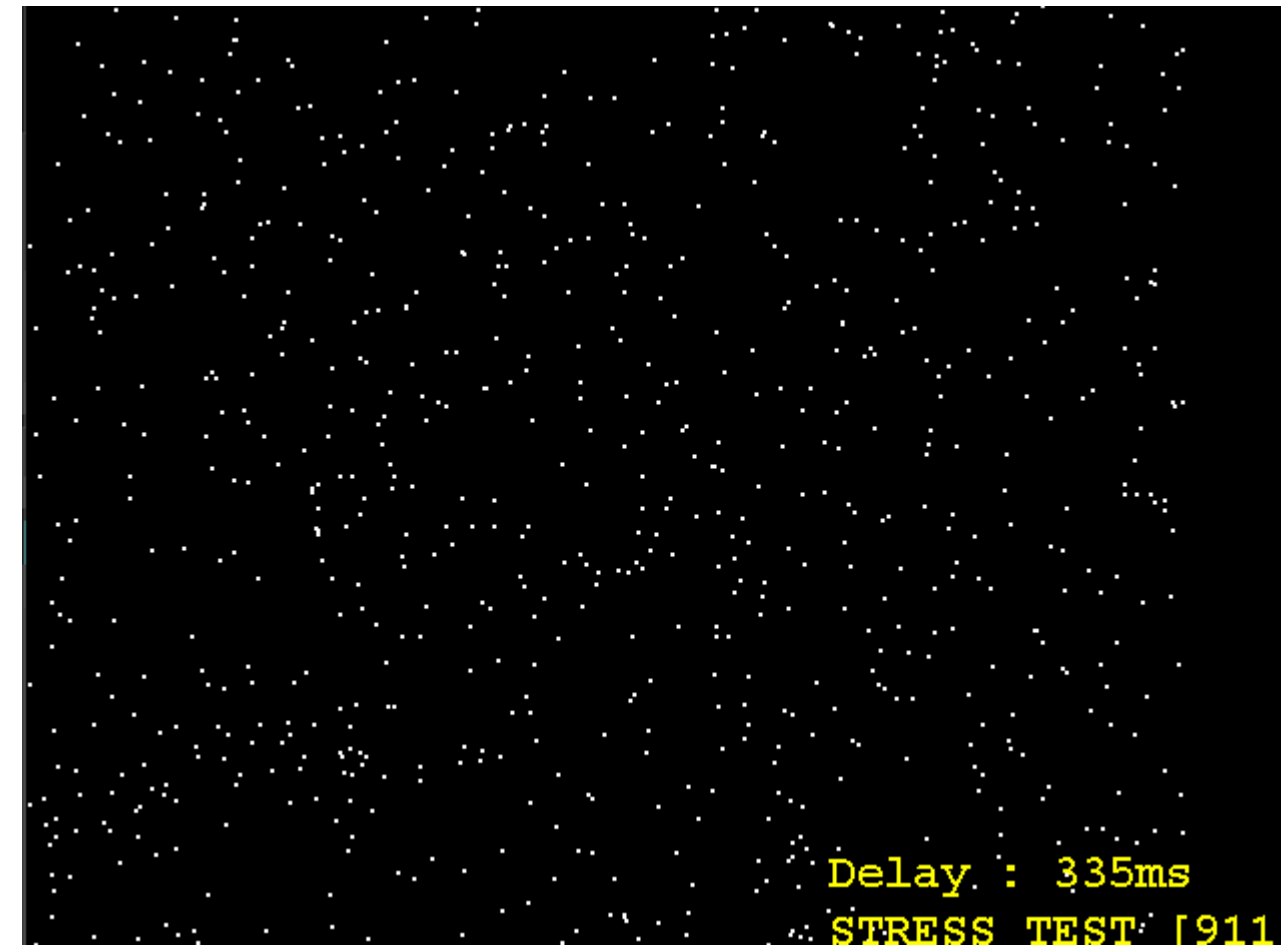
2000 x 2000크기의 게임 월드, 몬스터 20만 마리

SIMPLEST MMORPG – Performance Improvement

병목분석을 통한 성능 개선 – **시야처리** 및 **공간 분할**을 통한 최대 동접 증가



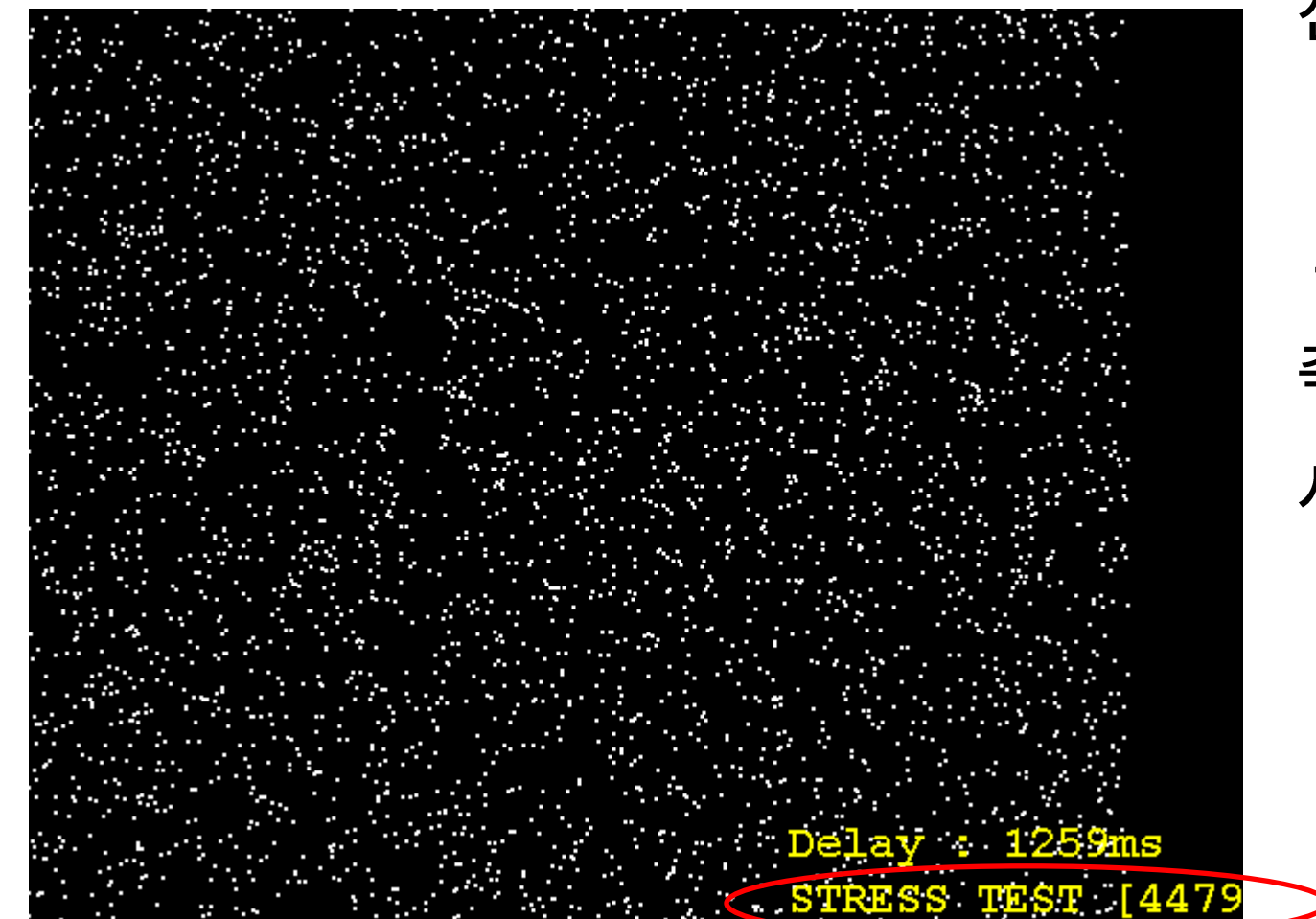
아무것도 적용하지 X



공간분할 적용



시야처리 적용



시야처리 + 공간분할

기존에는 캐릭터의 이동이나 상태 변화가 발생할 때 주변 플레이어를 탐색하기 위해 넓은 범위의 오브젝트 및 맵을 반복적으로 순회했습니다. 접속자 수가 증가할수록 시야 대상 탐색 비용과 불필요한 패킷 전송량이 함께 증가하여 서버의 성능을 낮추는 부분이었습니다.

이를 개선하기 위해 2000 x 2000의 게임 월드를 일정한 크기의 영역으로 나누는 공간 분할을 적용하고 플레이어가 위치한 영역과 인접 영역만 탐색하도록 시야 처리를 적용하였습니다. 또한 플레이어별 시야 진입·유지·이탈 상태를 관리하여 실제로 서로를 볼 수 있는 대상에게만 이동 및 상태 변경 패킷을 전송하도록 구성했습니다.

그 결과, 전체 오브젝트를 대상으로 수행하던 탐색 연산을 주변 영역 중심으로 축소하고 시야 밖 플레이어에게 전송되던 불필요한 네트워크 트래픽을 줄여 서버의 최대 동접을 **약 4479**까지 향상시켰습니다.

※테스트 환경

CPU: AMD RYZEN 9 8940HX

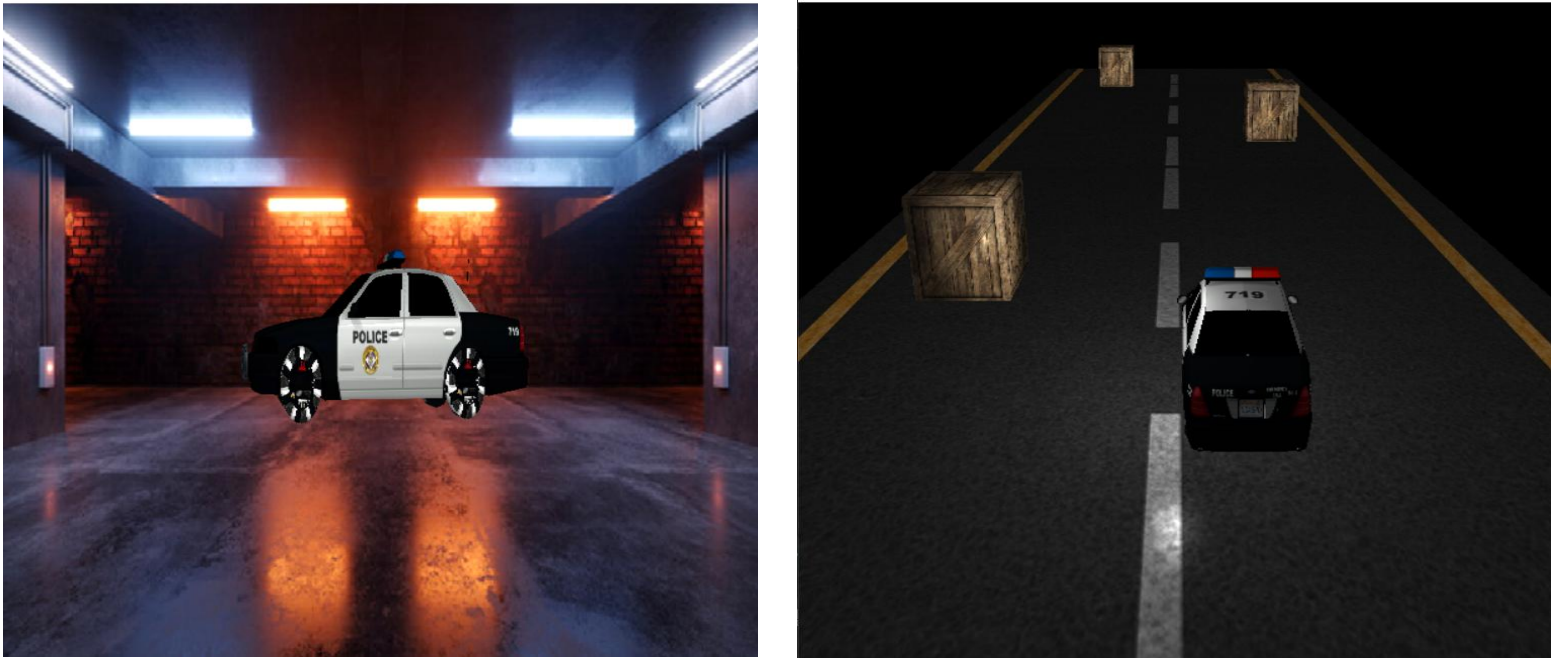
RAM: 64GB

2000 x 2000크기의 게임 월드, 몬스터 20만 마리

Other Projects

기타 프로젝트

2022학년도 2학기 컴퓨터 그래픽스 과목 텀프로젝트 – City Racing



게임 장르

3D 레이싱

개발 기간

2022.11.17 ~ 2022.12.15

개발 인원

클라이언트 프로그래머 2인

구현 내용

- 게임 프레임워크 및 맵 제작
- 카메라 기능 구현
- 이펙트 구현

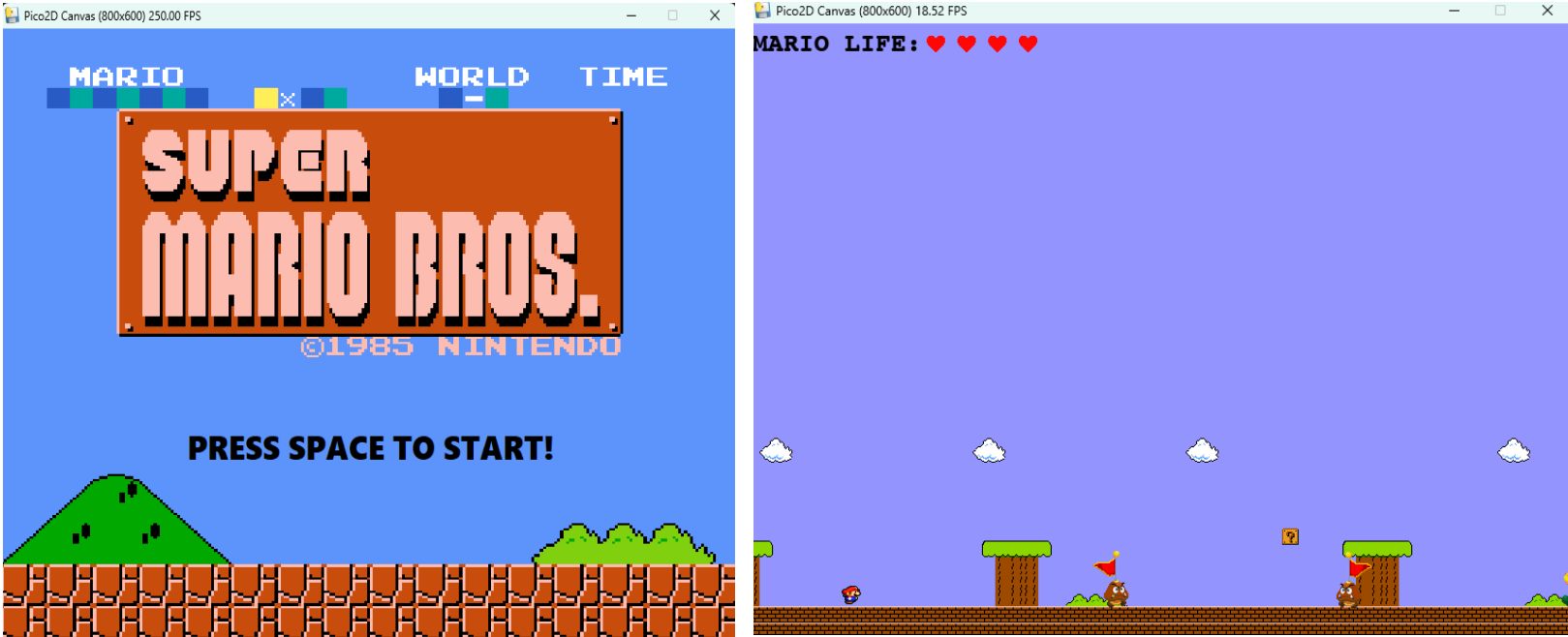
개발 언어

C++17

개발 도구 & 라이브러리

Visual Studio 2019, OpenGL, glm

2022학년도 2학기 2D 게임 프로그래밍 과목 개인 텀프로젝트 – Super Mario Game



게임 장르

액션, 캐주얼

개발 기간

2022.10.01 ~ 2022.12.09

개발 인원

1인

구현 내용

- 게임 프레임워크 및 맵 제작
- 캐릭터 이동 및 충돌처리 구현
- 몬스터 FSM, 아이템 구현

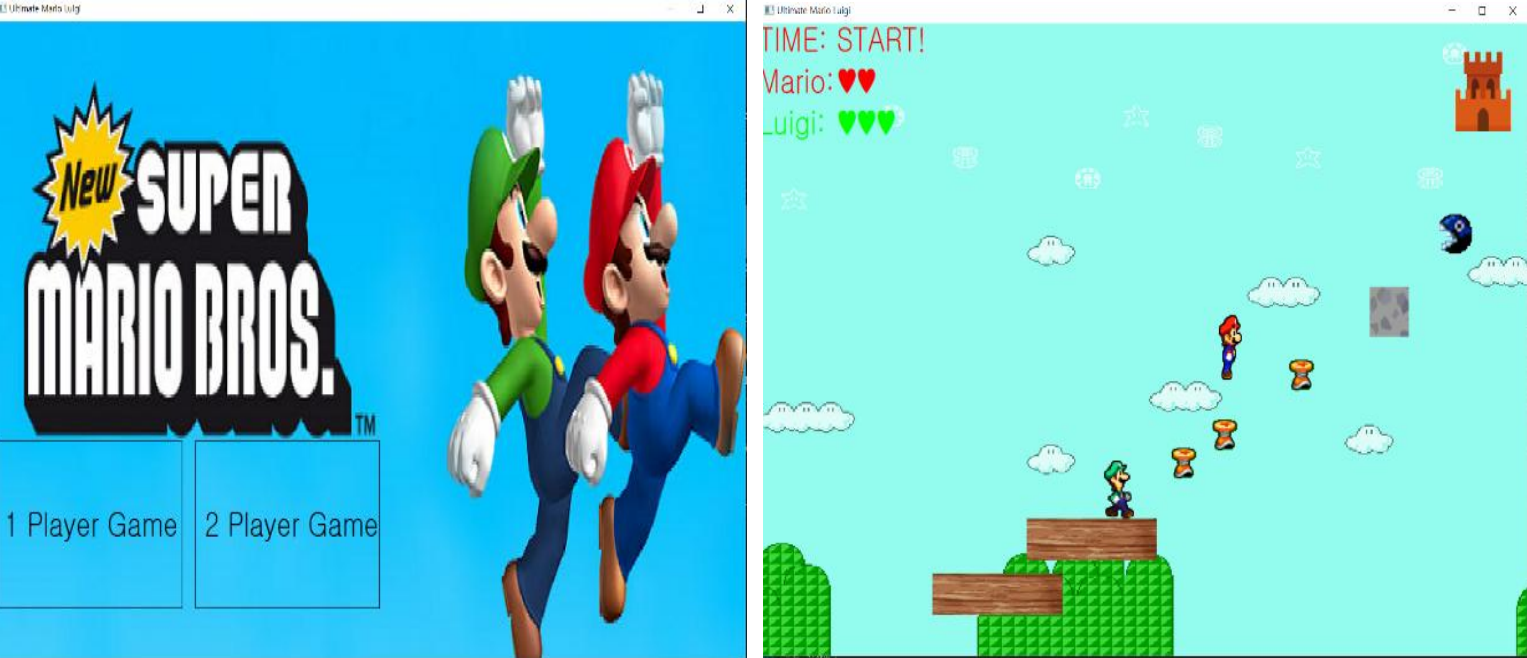
개발 언어

Python 3.10.6

개발 도구 & 라이브러리

Visual Studio Code, Pico2D

2022학년도 1학기 윈도우 프로그래밍 텀프로젝트 – Ultimate Mario & Luigi



게임 장르

파티 비디오, 캐주얼

개발 기간

2022.05.11 ~ 2022.06.15

개발 인원

클라이언트 프로그래머 2인

구현 내용

- 게임 프레임워크 및 맵 제작
- 장애물/아이템 설치 기능 구현
- 캐릭터 이동 및 애니메이션 구현

개발 언어

C

개발 도구

Visual Studio 2019

읽어주셔서 감사합니다